

Prompt-based automation of building code information transformation for compliance checking

Fan Yang^{a,*}, Jiansong Zhang^{a,b}

^a Automation and Intelligent Construction (AutoIC) Lab, School of Construction Management Technology, Purdue University, West Lafayette, IN 47907, USA

^b Digital Twin & Robotic Automation Center for Knowledge Sharing, Entrepreneurship, and Research (DigiTRACKER), Purdue University, West Lafayette, IN 47907, United States of America

ARTICLE INFO

Keywords:

Building code
Information transformation
Automated compliance checking
Prompt engineering
Natural language processing
Large language models (LLMs)

ABSTRACT

Transforming building code information into a machine-processable format is essential for automated compliance checking, yet it presents significant challenges. A prompt-based framework was developed to automate the conversion into a logic programming language. Its effectiveness was assessed by testing the framework on 51 requirements from the International Building Code (IBC) 2015, achieving 97.37 % precision and 95.88 % recall at the logic clause level, with only 2 % of the data used for training. Further testing on crash report transformation enhanced efficiency, reducing the average code generation time to approximately 60.8 s, thereby achieving a 27.8 % time savings compared to existing rule-based methods. This paper contributes to the body of knowledge by introducing an effective, versatile, and user-friendly approach to automated building code information transformation, markedly decreasing the reliance on training data, time, and manual efforts.

1. Introduction

1.1. Background context

Automated building code compliance checking (ACC) in the architecture, engineering, and construction (AEC) domain refers to the use of advanced computing technologies and algorithms to automatically verify whether building designs adhere to various codes, standards, and regulations. It significantly reduces the time and effort required for compliance checking, traditionally a manual and time-intensive process [1,53]. Achieving full automation in ACC requires a comprehensive approach encompassing four key stages: the representation of building codes, the representation of design information, the execution of checking rules, and the exportation of results [2].

The first stage, building code representation, is particularly crucial for the efficacy of ACC [58]. This process entails translating building codes, which are expressed in natural language, into a computable format [52,54]. This translation is pivotal for several reasons. Firstly, it ensures that the diverse and intricate regulations, which can vary widely in language and structure, are standardized and machine-readable. This standardization is essential for the accurate and consistent application of regulatory rules across different projects and jurisdictions. Moreover,

effective building code representation allows for the integration of these codes into compliance checking systems. This not only streamlines the compliance checking process but also significantly reduces the potential for human error, which is common in manual compliance checking. Lastly, by converting building codes into a format that ACC systems can interpret and apply, building code representation forms the foundation upon which the subsequent stages of design information representation, rule execution, and result exportation can be built [63,64,68]. Without it, the automation of building code compliance checking would be impractical, as the systems would lack the necessary framework to understand and apply the complex regulations that govern building design and construction and manually converting the rules are cost prohibitive.

1.2. Problem statement

Building code representation is challenging primarily due to the complex and diverse nature of building codes, which are extensive, technical, and vary widely across different regions and countries [3]. This complexity is compounded by the fact that building codes are not static; they evolve over time with new standards and practices [4]. This variability demands a high level of customization and adaptability in the representation process. To address these challenges, two primary

* Corresponding author.

E-mail addresses: yang2051@purdue.edu (F. Yang), zhan3062@purdue.edu (J. Zhang).

approaches have been employed: machine learning [5–7] and rule-based approaches [8–10,54,55,67]. The machine learning approach utilizes algorithms that learn from data, adapting to the complexities and nuances of building codes through exposure to various examples [5]. This method is particularly effective in handling unstructured data and improves as it processes more data, offering flexibility in managing different types of regulatory documents. However, its reliance on extensive annotated data for training presents a significant limitation, particularly where such data is scarce or costly to prepare. On the contrary, the rule-based approach utilizes expert-compiled rules and logic to ensure precision and reliability, especially in scenarios with well-defined regulations [11]. This method's strength lies in its ability to deliver consistent and accurate compliance checking outcomes, well founded on established rules. Yet, the requirement for substantial manual effort in rule development and maintenance is its primary drawback. Comparative analyses between these methods reveal a trade-off: rule-based systems excel in precision and recall within well-defined contexts (e.g., 96.9 % vs. 93.1 % in precision and 94.4 % vs. 92.9 % in recall) [12], whereas machine learning offers adaptability with less need for manual intervention, albeit with the challenge of securing adequate annotated data. Therefore, there is a need for a solution that can combine the precision and reliability of rule-based approaches with the flexibility and adaptability of machine learning methods.

The advent of Large Language Models (LLMs) provides a novel potential avenue for achieving comprehensive building code information transformation, effectively integrating the strengths of both machine learning and rule-based approaches. Among these, Generative Pre-trained Transformers (GPT) stand out as one of the most advanced methodologies within LLMs, with applications in fields such as natural language understanding [13], text generation [14], language translation [15], and even content creation in art and music [16]. GPT models, trained on vast datasets, exhibit an unparalleled ability to understand and generate human-like text [74]. This attribute renders them exceptionally suitable for decoding the complex and nuanced language inherent in building codes. Besides, LLMs can maintain contextual understanding over long text passages, ensuring comprehensive and consistent interpretation of regulatory requirements. Moreover, the adaptability and scalability of LLMs are significant strengths, allowing these models to be fine-tuned with additional data specific to regional and contextual differences in building codes. This adaptability makes LLMs suitable for supporting the encoding of diverse and evolving regulatory requirements in automated code compliance checking (ACC). By automating the translation process, LLMs could drastically reduce the reliance on extensive manual annotation or the meticulous development of rules that would characterize traditional machine learning and rule-based approaches, respectively. Additionally, when enhanced through prompt engineering and fine-tuning, GPT models can be tailored to the specific nuances and requirements of building codes. This bespoke approach effectively marries the precision and reliability of rule-based methods with the adaptability and scalability of machine learning methods, offering a sophisticated but potentially highly efficient solution for building code information transformation.

In previous research, Zhang [12] investigated the application of a GPT model for translating building code requirements into a programming language, comparing its efficacy against a leading rule-based method. The comparison was nuanced due to the different target languages (Python for GPT vs. B-Prolog for the rule-based approach), primarily focusing on evaluating the GPT model's ability to comprehend building regulation logic and the transformation time of each approach. While this study illuminated the potential of GPT models in understanding and converting building codes, it left the model's comprehensive performance in this domain largely unexamined. Thus, a significant gap in knowledge remains regarding the full spectrum of GPT's capabilities, when enhanced with prompt engineering, in accurately and efficiently translating complex building codes into programmable formats.

1.3. Research gaps, research questions, and objectives

The current study identifies a critical gap in ACC. Existing methods struggle with the complexity and diversity of building code requirements, which vary widely across regions and evolve over time, posing significant challenges in their encoding into computable formats. To automate such encoding, two main approaches are machine learning-based and rule-based, respectively. However, machine learning typically requires extensive annotated data, whereas rule-based methods demand significant manual effort for rule development and maintenance. This highlights the need for a method that combines and synergizes both approaches, capitalizing on the precision and reliability of rule-based approaches, and the flexibility and adaptability of machine learning techniques. Additionally, the full potential of GPT models, especially when enhanced with prompt engineering, has not been comprehensively examined for translating complex building codes into machine-readable formats.

This research seeks to address the following key questions:

1. How can the strengths of machine learning and rule-based approaches be effectively combined to improve building code representation for ACC?
2. How good are GPT models, when enhanced with prompt engineering, in translating complex building codes into machine-readable formats?
3. How much improvement in terms of accuracy and efficiency can be achieved compared to pure rule-based or machine learning-based methods?

To address these gaps and questions, the study has set the following objectives:

1. Develop a novel framework leveraging GPT models and prompt engineering for automating the transformation of building codes into machine-readable formats.
2. Evaluate the framework's effectiveness and versatility through empirical testing.
3. Compare the proposed framework's performance with existing methods in terms of accuracy, efficiency, and the extent of manual input required.

By addressing these research gaps, questions, and objectives, this study aims to advance the field of ACC through the development and evaluation of a novel, integrated approach leveraging advanced GPT models and prompt engineering.

1.4. Proposed framework

In this paper, a new framework leveraging prompt engineering and the GPT model is introduced for full automation of building code information transformation. This framework consists of two main components: a front-end, executable file-based user interaction (UI) module for reading building code requirements, user instructions, and saving the generated programming script, and a back-end information conversion (IC) module for executing user instructions, information conversion and processing. Prompt engineering involves designing precise input prompts to direct the model's output, customizing the responses to accurately capture the intricate and nuanced language of building codes [57]. This approach allows the GPT model to effectively interpret and translate building regulations into machine-readable formats.

In order to evaluate the effectiveness and versatility of the proposed framework, it was tested on the information transformation of international building code and car crash reports and compared with manual approach and the state-of-the-art methods. The information transformation of crash reports helps extract key information of traffic crashes and further support automated crash reconstruction, which

provides an approach for safety testing of autonomous cars in an efficient and risk-free way. The evaluation results demonstrate the framework's capability to accurately convert information with fewer manual inputs and learning examples compared to traditional rule-based and machine learning-based methods, while delivering performance on par with existing methods in accuracy and efficiency.

1.5. Contributions and originality

The research makes substantial theoretical, practical, and methodological contributions. Theoretically, it introduces an innovative approach to ACC by integrating GPT models with prompt engineering, demonstrating advancements in natural language processing within the AEC domain. This integration effectively combines the strengths of both machine learning and rule-based methods, showcasing the potential of GPT models to understand and process the complex and nuanced languages of building codes. Practically, the framework enhances efficiency and accuracy in ACC by significantly reducing the need for extensive manual annotation and meticulous rule development. It streamlines the building code compliance checking process and demonstrates versatility by applying the framework to both building codes and car crash reports, indicating its potential for broader applications in various compliance and safety contexts. Methodologically, the study presents a novel two-component framework comprising a front-end user interaction module and a back-end information conversion module. This design facilitates user interaction and efficient information conversion. The framework's evaluation through comprehensive testing and comparison with manual approaches and state-of-the-art methods provides a robust assessment of its performance.

The originality of the proposed framework lies in its achievement of complete automation in transforming building codes into machine-readable formats. The seamless integration between the user interaction and information conversion modules, processing natural language inputs into executable programming scripts, represents a significant advancement in ACC. Furthermore, the framework's ability to accurately convert information with fewer manual inputs and learning examples compared to traditional methods underscores its innovative efficiency.

This paper is structured as follows: [section 2](#) provides the related background information, [section 3](#) introduces the methodology of this research, [section 4](#) presents the detailed experiments, [section 5](#) analyzes the experimental results and evaluate the performance of the proposed framework, [section 6](#) highlights the contribution and practical benefits of this research, [section 7](#) concludes this paper, and finally [section 8](#) discusses limitations and future work.

2. Background

2.1. Information transformation in ACC

In the field of Natural Language Processing (NLP), information transformation stands as a novel technique to facilitate the conversion of unstructured textual data into a structured computable format. This process is pivotal for extracting meaningful insights and enabling computational systems to understand and manipulate natural language data effectively. By translating the nuances of natural language into a structured representation, NLP systems are equipped to perform a myriad of tasks, including but not limited to, data analysis [17], information retrieval [18], and the generation of actionable knowledge [19]. This transformative process not only enhances the accessibility of textual data for computational analysis but also serves as a bridge between the ambiguity of natural language and the precision required for algorithmic processing. The current information transformation in ACC encompasses a broad spectrum of methodologies using two main approaches: rule-based parsing and machine learning-based parsing.

The rule-based approach to information transformation involves the

use of a set of predefined, deterministic rules to convert unstructured textual data into a structured format. This approach is grounded in linguistic theories and relies on the explicit programming of language rules which are typically crafted by experts to parse and understand texts. Rule-based systems identify specific patterns, keywords, and syntax within the target text, applying these rules to extract and structure information accordingly. The mechanism of a rule-based system typically involves lexical analysis, syntactic parsing, and semantic interpretation, each step governed by a comprehensive set of rules designed to capture the complexity and variability of natural languages. Thanks to this, rule-based systems are highly precise for the scenarios they are designed to handle, and the deterministic nature of rule-based systems provides clear traceability and explainability of the decisions made during the information transformation process. In addition, rule-based systems do not require large volumes of annotated data for training, making them suitable for applications with limited data availability. However, the development and maintenance of rule-based systems can be labor-intensive, as it requires extensive domain expertise to create and update. As a result, these systems may struggle to adapt to new, unseen examples that do not fit the predefined rules (i.e., the patterns of which were not seen during training). Since natural language is inherently ambiguous, rule-based systems may find it challenging to interpret text with multiple meanings without sophisticated disambiguation rules. In the domain of ACC for building codes, some research has been conducted to use the rule-based approach to translate the complex, often legalistic language of building regulations into actionable, machine-readable formats [20,65]. For example, Zhang and El-Gohary [11] introduced a first-order logic (FOL)-based schema that achieves complete automation and reasoning in regulatory compliance of building designs, using ontology-based logic clauses. It reports 100 % recall and precision with perfect information and 98.7 % recall and 87.6 % precision with imperfect information in detecting noncompliance within building designs. Zhou and El-Gohary [9] proposed an Ontology-Based Information Extraction (OBIE) algorithm that automates the extraction of energy requirements from energy conservation codes with a 97.4 % recall and a 98.5 % precision, showcasing the use of ontology-based pattern matching and sequential dependency methods to handle complex provisions and reduce ambiguities. These examples underscore the effectiveness of rule-based systems in specialized domains where precision and clarity are paramount, demonstrating their potential to transform complex legal texts into actionable insights with high accuracy. Nevertheless, the ongoing need for expert input and the challenge of adapting to evolving language and regulations highlight the dynamic balance between the robustness of rule-based approaches and the flexibility required for their sustained usability in changing environments.

Unlike rule-based systems, machine learning-based approach to information transformation enables computers to learn from data, identifying patterns, relationships, and structures within text through statistical methods and artificial intelligence (AI) techniques. The core mechanism of machine learning-based approaches involves several key stages: data preprocessing, feature extraction, model training, evaluation, and prediction. Compared to rule-based approach, machine learning-based approach can automatically adapt to changes in language use or domain-specific nuances without manual updates to rules. Also, the model can handle large volumes of data more efficiently, making them suitable for applications with extensive datasets. However, machine learning-based approach usually requires a large amount of labeled data, and the performance of models is heavily dependent on the quality of the training data [84]. Another disadvantage of machine learning-based approach is that machine learning models, especially deep learning models, can act as "black boxes", making it difficult to understand how decisions are actually made. Researchers have explored using machine learning models to understand the semantic content of building codes and automatically or semi-automatically map these requirements to features in BIM [60,66]. For instance, Zhang and El-Gohary [5] proposed a deep learning-based method that uses transfer

learning strategies to learn from both AEC regulatory documents and the Penn Treebank data to further support deep information extraction from regulatory documents and automated compliance checking. This method addresses the problem that a large number of annotated training datasets are unavailable in the AEC field and achieves fully automated information extraction. Zhong et al. [6] introduced a novel deep learning approach using a combination of the bidirectional long short-term memory (Bi-LSTM) and the conditional random field (CRF) models to extract and structure construction procedural constraints from regulations. The results demonstrate the model's applicability and generalizability to various information extraction tasks within similar domains by showcasing successful implementation and validation using national standards. These instances highlight the adaptability and scalability of machine learning approaches in dealing with complex, unstructured data, offering promising avenues for automating and enhancing the accuracy of compliance checks in the construction industry. Yet, the reliance on substantial, high-quality training datasets and the challenge of interpreting model decisions underscore the need for ongoing research to look into making these technologies more accessible and explainable for users across diverse fields.

Table 1 summarizes current different information transformation tasks within AEC industry, specifically focusing on the extraction and reasoning of information from various regulatory and design documents. The rule-based approaches in the table are precise but are limited by their reliance on extensive domain expertise for rule formulation (Limitation 1) and suffer from difficulty in adapting to new or varied scenarios not covered by existing rules (Limitation 2). Machine learning-based methods, while flexible and capable of learning from data, suffer from requiring large amounts of high-quality annotated data (Limitation 3 and 5), and lack transparency and explainability in decision-making (Limitation 4). LLMs could represent a new frontier for these tasks, potentially mitigating some of these limitations. Leveraging vast amounts of textual data and sophisticated neural network architectures, LLMs have the capability to understand, generate, and transform language in ways that were previously difficult to achieve with traditional rule-based or even conventional machine learning and deep learning-based approaches. The subsequent section details GPT models.

2.2. Generative pre-trained transformer models

As a subfield of Artificial Intelligence (AI), NLP stands at the forefront of bridging the gap between human language and computer understanding, offering a fascinating field where technology intersects with linguistics. This field encompasses a variety of tasks aimed at

enabling "machines to understand, interpret, and generate human language in a way that is both meaningful and contextually relevant" [75].

Among the most notable developments in NLP in recent years are Large Language Models, such as GPT series developed by OpenAI, and BERT (Bidirectional Encoder Representations from Transformers) by Google. These models, characterized by their vast number of parameters and deep learning architectures, have set new benchmarks in the field's capability to handle complex language tasks. LLMs have demonstrated remarkable proficiency in generating human-like text [22], understanding context [23], and even performing specific tasks like coding [24], composing music [25], and creating drawings based on text descriptions [26]. The versatility of LLMs in these diverse domains is significantly enhanced by prompt engineering, a technique that allows users to craft inputs that guide the model towards understanding the intended task, shaping both the form and substance of its outputs. This customization enables LLMs to apply their general capabilities to specific challenges. Moreover, the iterative nature of prompt engineering allows for a dynamic interaction with LLMs, where the outputs can be refined through successive prompts based on initial results. This iterative feedback loop is invaluable in fine-tuning the model's performance, ensuring that the final output meets the user's expectations and requirements.

Despite the successful application of LLMs across many fields, to the best of the authors' knowledge, there is a lack of published research that thoroughly examines the use of a prompt-based approach for the transformation of building code information in AEC domain. Therefore, further investigation into this subject is imperative to elucidate the capabilities of LLMs in revolutionizing the accessibility, interpretation, and conversion of building codes and regulations into machine-comprehensible formats, to address this important problem that has prevented wide adoption of ACC in practice.

2.3. Logic programming language

Logic programming languages are a type of programming language that is based on formal logic. "Any program written in a logic programming language is a set of sentences in logical form, expressing facts and rules about some problem domain" [76]. These languages are primarily concerned with the use of logic to describe the problem (or task) to be solved, rather than detailing the process to solve it. The roots of logic programming can be traced back to the early research in artificial intelligence and computational logic [27]. The development of these languages was heavily influenced by the work on automated theorem proving and symbolic computation [28]. Logic programming languages

Table 1
Current different information transformation tasks in AEC.

Information processing task	Approach type	Data source/document type	Utilized technologies/tools	Limitations
Regulatory information representation and reasoning [11]	Rule-based	Construction regulatory (IBC) and design information	First-order logic, B-prolog	1,2
Regulatory document data extraction [5]	Machine learning-based	AEC regulatory documents (IBC, IECC, and ADA Standards) and the Penn Treebank data	Deep learning model, transfer learning	4,5
Qualitative procedural constraints extraction [6]	Machine learning-based	Construction regulatory information (Code for Acceptance of Construction Quality of China)	deep neural network	3,5
Energy requirements extraction [9]	Rule-based	Energy conservation codes (International Energy Conservation Code)	ontology, GATE	1,2
Construction procedural document information extraction [10]	Rule-based	The Construction Specifications Institute (CSI) specifications	ontology	1,2
Spatial regulation interpretation [21]	Rule-based	utility regulations (Indiana State Utility Accommodation Policy)	ontology	1,2
Safety regulation data extraction [7]	Machine learning-based	Occupational Safety and Health Administration (OSHA)	BiLSTM-CNN-based model, Transfer learning	4,5

1 - Requires a lot of manual effort.

2 - Limited scalability and flexibility.

3 - Requires large amounts of annotated data.

4 - Lack of interpretability.

5 - Dependency on data quality.

are suited to a variety of applications, particularly those involving complex symbolic reasoning, such as database management and querying [29], constraint satisfaction problems [30], and artificial intelligence and expert systems [31].

One of the foundational languages in this category, Prolog (Programming in Logic), was created in the 1970s and became a standard bearer for logic programming [27]. It is based on the concepts of predicates, facts, and rules. A Prolog program consists of a series of facts and rules about objects and their relationships, and queries are made to solve problems by inferring new facts and rules from the existing ones. As an example, Picat is a multi-paradigm programming language that is rooted and developed in Prolog [32]. In Picat, facts are used to represent knowledge in a straightforward way. For example, in a compliance checking system, logic facts can represent buildings' characteristics or code requirements:

```
building("OfficeBuilding", height, 70). % Height in meters.
building("ResidentBuilding", height, 20). % Height in meters.
code_requirement(max_height, 60). % Height in meters.
```

These logic facts state that:

```
Building "OfficeBuilding" has a height of 70 m.
Building "ResidentBuilding" has a height of 20 m.
The required maximum building height in the building code is 60 m.
```

Logic rules can also be defined to check if the building complies with the code requirements, for example:

```
height_compliance(Building) =>
building(Building, height, Height),
code_requirement(max_height, MaxHeight),
Height = < MaxHeight.
```

In the rule, "*height_compliance(Building)*" is the head, which defines the rule's name (*height_compliance*) and its parameter (*Building*). Because capitalized words represent variables in Picat, the parameter serves as a placeholder for the building's name or identifier for representing the object that needs to be checked for height compliance. The body of the rule is everything after the "*=>*" operator (i.e., the body of the logic rule) and consists of three parts, each separated by commas, which represent conditions that must all be evaluated to true for the rule to apply. The first part "*building(Building, height, Height)*" retrieves the height (*Height*) of the specified building (*Building*). It matches against logic facts in the database that represent buildings' heights. The fact must have the building's name or identifier as the first argument, the attribute being checked (*height*) as the second argument, and the actual height value (*Height*) as the third argument. Similarly, the second part "*code_requirement(max_height, MaxHeight)*" retrieves the maximum allowed height (*MaxHeight*) from the building code requirements stored as facts in the database. The third part "*Height = < MaxHeight*" is a comparison operation that checks if the building's height (*Height*) is less than or equal to the maximum allowed height (*MaxHeight*). If all conditions are met, it means the building complies with the height requirement.

Queries are used to ask the system if the building meets specific code compliance criteria. For example, to check if the "*OfficeBuilding*" complies with the height requirement, the following query can be used.

```
?height_compliance("OfficeBuilding").
```

If all conditions in the rule's body are satisfied - meaning the building's height does not exceed the maximum allowed height - the rule concludes that the building complies with the height requirement, and the query will return true (or success). If any condition fails (e.g., if the building's height exceeds the maximum allowed height), the rule concludes that the building does not comply with the height requirement, and the query will return false (or fail).

Another important concept in programming language is a predicate, which is a fundamental concept that represents a relation among certain terms. Each predicate is defined by its name and the number of arguments (arity) it takes. For example, in the logic fact "*building("OfficeBuilding", height, 70).*", "*building*" is a predicate with arity 3, indicating an entity and its attributes. In other words, predicates form the basis of

expressing both the static knowledge (logic facts) and dynamic inferences (logic rules) in a logic program, while queries allow for the extraction of knowledge by leveraging the interplay between logic facts and logic rules through the predicates they involve.

As an implementation and further development of Prolog, Picat [32] inherits Prolog's declarative nature, which allows programmers to specify what the program should accomplish rather than how to accomplish it. However, Picat extends Prolog's capabilities with syntax enhancements, a rule-based system for describing algorithms, and built-in support for solving constraint satisfaction problems, making it more versatile and easier to use for a broader range of applications, such as constraint solving and optimization, artificial intelligence, and automated reasoning [32]. In addition, Picat's blend of declarative programming, pattern matching, and constraint solving, along with its ease of use and extensibility [32], make it an attractive choice for developing systems focused on automated reasoning, such as automated compliance checking. Its capabilities allow developers to effectively model complex regulatory requirements and automate the process of compliance checking across various domains.

2.4. Text similarity measurements

Text similarity in NLP refers to the process of determining how closely two pieces of text are related in terms of their meaning or content. This concept is crucial in various NLP tasks because it allows computers to process and understand human language in a way that can be quantitatively measurable. Text similarity can be used in NLP filed for information retrieval [33,34], plagiarism detection [35], questions answering systems [36,59], sentiment analysis [37] and other tasks.

Among them, one of the most important applications is in evaluating the quality of machine translation. Machine translation is the process of using computer algorithms to translate text or speech from one language to another automatically. It also includes translating natural language into programming code, which is usually known as code generation [38,39]. When machine translation is applied to the domain of code generation, the evaluation criteria changed a bit compared to traditional language-to-language translation. The primary goal remains to assess whether the generated code accurately reflects the intent expressed in the natural language description. However, due to the structured and formal nature of programming languages, the evaluation focuses more on the functional correctness, translating efficiency, and readability of the generated code [40]. In the following paragraphs, three pivotal distance-based text similarity metrics - Levenshtein Distance, Jaro-Winkler Similarity, and Jaccard Distance - are introduced as foundational measurements for assessing the text alignment of translated code in this study.

"The Levenshtein distance, also known as the edit distance quantifies the minimum number of single-character edits (insertions, deletions, or substitutions) required to transform one string into another." [41,77]. In the context of code generation, Levenshtein Distance can be used to assess the syntactic accuracy of generated code compared to a reference implementation. A smaller distance indicates that the generated code is syntactically closer to the reference, with a Levenshtein Distance of zero denoting identical code. The larger the Levenshtein Distance is, the larger the difference between the two is. For example, The Levenshtein Distance between "brick" and "block" is two because two single-character edits (e.g., substitution of "r" in "brick" with "l" to get "blick" and substitution of "i" in "blick" with "o" to get "block") are needed to transform one to the other. Thus, the Levenshtein Distance offers a quantifiable and straightforward method for evaluating the similarity between texts or code snippets, making it an invaluable tool in the evaluation of code generation.

The Jaccard Distance is a metric used to measure the dissimilarity between two sets [42]. It is defined as one minus the Jaccard similarity coefficient, which is "the size of the intersection divided by the size of the union of the sample sets" [78]. The formula to calculate the Jaccard

Distance is:

$$JD(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

where A and B are two sets of code snippets in the context of code generation. For example, there are two codes listed below:

```
str1 = "printf('hi')".
str2 = "print('hi')".
```

the size of the first string (the number of the unique characters in the first string) is ten (i.e., {'h', 'r', '(', '(', 'n', 'f', 'i', ')', 'p', 't'}), while the size of the second string is nine (i.e., {'h', 'r', '(', '(', 'n', 'i', ')', 'p', 't'}). Therefore, the size of the intersection of string 1 and string 2 is nine, and the size of their union is ten. Then the Jaccard Distance of these two strings can be calculated as:

$$JD(str1, str2) = 1 - \frac{9}{10} = 0.1 \quad (2)$$

Following the equation of the Jaccard Distance, it can be seen that the Jaccard Distance range from zero to one (zero represents complete similarity between the sets, and one represents complete dissimilarity, with no shared elements) and the order of characters in each set does not affect the Jaccard Distance. Therefore, the Jaccard Distance can be useful for assessing the overall structure and content of the generated code in relation to a reference codebase, focusing on the presence or absence of certain lines rather than their order.

Another distance-based metric to evaluate the code generation quality is the **Jaro-Winkler Similarity** [43]. It measures the similarity between two strings by considering the number and sequence of shared characters, as well as the number of transpositions [79]. Jaro-Winkler Similarity is an extension of the Jaro Similarity, which is a statistical method for measuring the similarity between two strings, especially for short strings. The Jaro Similarity score between two strings is defined as:

$$J(A, B) = \frac{1}{3} \left(\frac{m}{|A|} + \frac{m}{|B|} + \frac{m-t}{m} \right) \quad (3)$$

where $|A|$ and $|B|$ are the lengths of strings A and B , respectively. The variable m is the number of matching characters [80]. Two characters from A and B are considered matching if they are the same and not farther than $\left(\left\lceil \frac{\max(|A|, |B|)}{2} \right\rceil - 1 \right)$ positions apart in their respective strings [80]. The variable m determines the maximum distance characters can be apart to be considered a match. It not only ensures that comparisons are only made between characters that are reasonably close to each other in the string, but also allows for some flexibility in character positions, making sure some common discrepancies, such as small typos, do not significantly change the strings' similarity. The variable t is half of the number of transpositions [80,81]. A transposition occurs "when two matching characters are in a different order in the two strings" [81]. The total number of transpositions is divided by 2 as per the original Jaro definition [71] since each transposition involves two characters.

Extended on the Jaro Similarity, the Jaro-Winkler Similarity gives more weight to prefixes, under the assumption that errors are more likely to occur at the ends of strings. It is defined as:

$$JW(A, B) = J(A, B) + \{l \cdot p \cdot [1 - J(A, B)]\} \quad (4)$$

Where l represents the length of the common prefix at the beginning of the strings, capped at a maximum of 4 characters, and p is a constant that determines how much the score is increased based on the presence of a common prefix (usually $p = 0.1$) [82,86].

The following example takes "brick" and "block" as two strings to illustrate how to calculate their Jaro-Winkler Similarity. The lengths of both strings are five, and matches are sought within $\left(\frac{5}{2} \right) - 1 =$

1 position. The matching characters are "b", "c", and "k", so $m = 3$. Transpositions are matching characters that are in a different order. Since all matching characters are in the same order in both strings, $t = 0$. The Jaro similarity score between two strings is:

$$J(A, B) = \frac{1}{3} \left(\frac{3}{5} + \frac{3}{5} + \frac{3-0}{3} \right) = \frac{11}{15} \quad (5)$$

For the Jaro-Winkler Similarity, "brick" and "block" share a prefix of "b", so $l = 1$ and with a chosen scaling factor $p = 0.1$, their Jaro-Winkler Similarity is calculated as follows:

$$JW(A, B) = \frac{11}{15} + 1 \cdot 0.1 \cdot \left(1 - \frac{11}{15} \right) = 0.76 \quad (6)$$

The Jaro-Winkler Similarity score ranges from zero to one: a score of one and zero indicates that the two strings are identical and completely dissimilar, respectively. A Jaro-Winkler Similarity score of 0.76 between two strings indicates a relatively high degree of similarity, taking into account both the matching characters (including their order and proximity) and the significance of a common prefix at the start of the strings. These calculations illustrate how the Jaro-Winkler Similarity builds on the Jaro Similarity score by emphasizing the importance of matching prefixes in assessing string similarities. The Jaro-Winkler Similarity can be used to evaluate the similarity between generated code and reference code, especially in contexts where certain patterns or structures are expected to be present at the beginning of the code snippets. It is also useful in scenarios where the correct ordering of characters is crucial, and slight deviations may still be acceptable.

In addition to the above three distance metrics, precision, recall, and the F1 score are widely used evaluation metrics in tasks that involve classification or prediction, such as code generation, where the goal could be to accurately generate code snippets based on natural language descriptions [12,61,62]. The formulas for calculating precision and recall are delineated as follows:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (7)$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (8)$$

$$\text{F1 Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

where True Positives (TP) is the number of correctly generated code snippets that match the expected output, False Positives (FP) is the number of code snippets that were incorrectly generated but do not match the expected output, and False Negatives (FN) is the number of relevant code snippets that the model failed to generate.

These metrics help in assessing the quality and reliability of code generation systems, guiding improvements and adjustments to better meet the desired outcomes. They are essential for understanding the strengths and weaknesses of a model, especially in tasks like code generation where both effectiveness and versatility are crucial.

3. Methodology

In this paper, the authors proposed a prompt-based framework using prompt engineering to automate the transformation of building code information into a designated logic programming language. The structured workflow of this framework is illustrated in Fig. 1.

3.1. Logical workflow

Initially, the process involves the creation of a GPT-based translator. Its principal role is to interpret building code requirements and translate them into the target logic programming language. To configure this

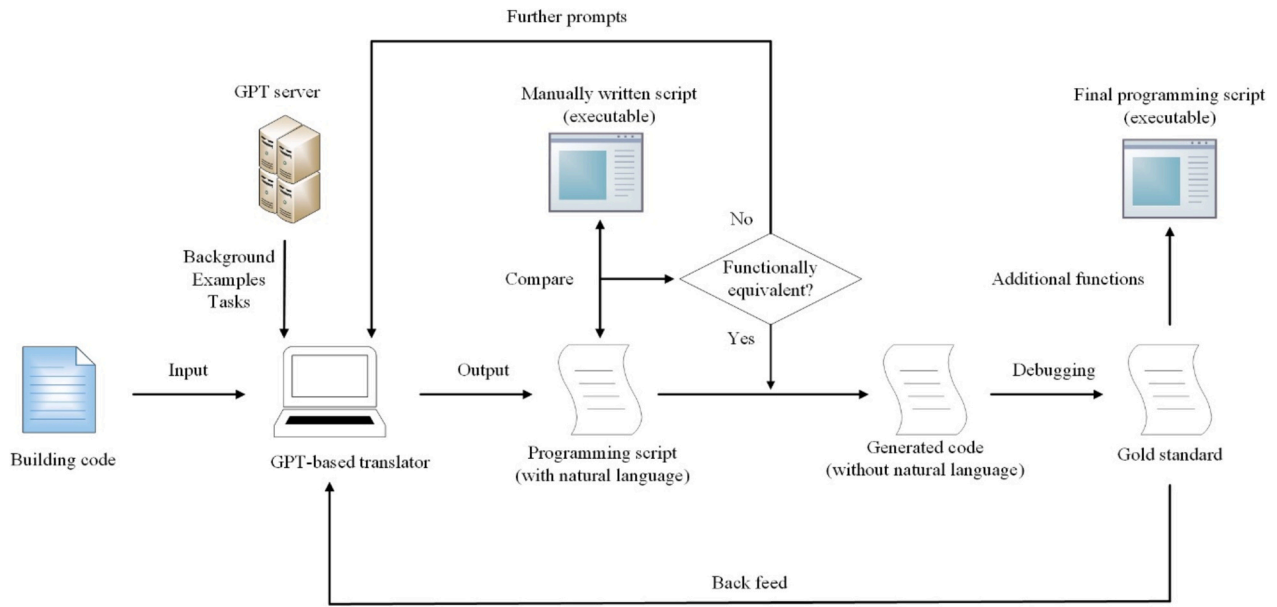


Fig. 1. Logical workflow of the proposed framework.

translator, the ChatGPT-4 model [72] is accessed through API and provided with foundational information encompassing the essential concepts and grammatical rules of the intended programming language, alongside examples of inputs (building code requirements) and their corresponding outputs (logic programming code). The logic programming code generated is designed to encapsulate basic concepts of the building code within logic facts and articulate the code's requirements through logic rules.

Following the establishment of the translator, it is tasked with translating new building code requirements based on specific task descriptions input into the system. The initial output from this translation process is termed as programming script with natural language, characterized by its inclusion of explanations and descriptions alongside the logic programming code. Concurrently, these new building code requirements undergo a manual analysis and translation into the target logic programming language, referred to as the manually written script in Fig. 1. The GPT-based translator's effectiveness is evaluated by comparing its generated logic programming code against manually translated code to determine their functional equivalence. Should discrepancies arise, additional prompts are required to refine the output until equivalence is reached. Upon achieving this milestone, the programming script, initially rich in natural language explanations and descriptions, is meticulously processed to extract a clean code version,

designated as the gold standard. This version is recognized as the benchmark for accuracy and effectiveness, representing the building code requirements optimally. To further enhance its utility, the gold standard is augmented with specific built-in functions that facilitate compliance checking, thereby producing the final programming script ready for execution on the logic programming language platform. Additionally, the gold standard is recycled back into the translator's training regime, serving as a vital component for the continuous improvement of its translation capabilities for future tasks.

3.2. Model creation

The logical workflow of the proposed framework is encapsulated within an executable file, with Fig. 2 illustrating the user's interactions with its two primary modules. Note that the user here is a developer user whose goal is to efficiently generate computable representation of building codes, instead of end users of ACC systems. This framework enables such users to input requests and prompts to the translator, from which they also receive responses. The user interaction module features a chat box for direct communication, a code save feature for archiving the final programming script, an example rules function for storing learning examples and gold standards, alongside background information and built-in functions. Requests from users are relayed by the user

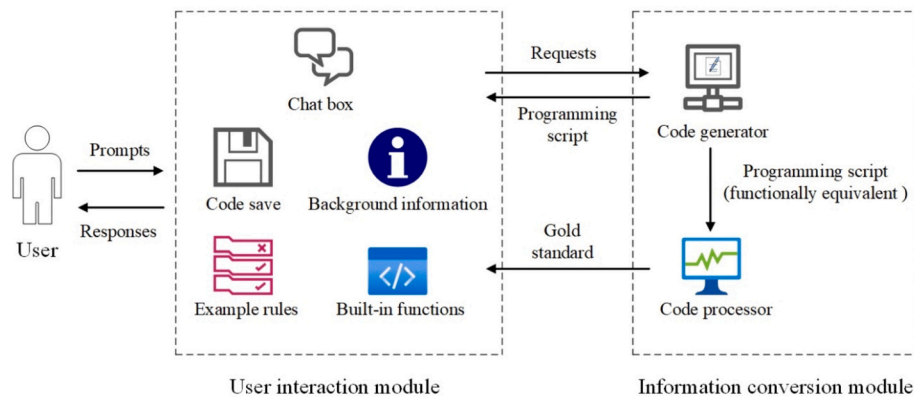


Fig. 2. Overview of the proposed framework.

interaction module to the code generator, a key component of the information conversion module. Here, the code generator assesses the requests, producing a corresponding programming script that is then returned to the user interaction module for a comparative verification against a manually written script. Upon achieving functional equivalence with the manually written script, the programming script is forwarded to the code processor unit to create the gold standard. This standard is then circulated back to the user interaction module, where it is preserved as example rules. Subsequent to the formulation of the gold standard, users can employ the code save function to refine it into an executable programming script, which can be stored locally on a computer.

3.3. Process flow

A UML diagram of the process flow is presented in Fig. 3. The workflow initiates with the creation of a GPT-based translator, which is pivotal to the framework. This translator is configured by providing it with foundational information, encompassing essential concepts, grammatical rules, and examples. Following the configuration, the translator undertakes the task of translating building code requirements into the target logic programming language, generating an initial programming script that includes logic facts and logic rules representing the building code requirements. To ensure the accuracy of the generated script, a comparison is made with a manually written version of the code. A critical decision point in the process involves checking for

functional equivalence between the generated code and the manually written code. If discrepancies are identified, the output undergoes refinement through additional prompts, and the translation process is repeated. This iterative refinement continues until functional equivalence is achieved. Once the generated code is verified as functionally equivalent to the manually written code, the process advances to removing natural language explanations from the code to create a clean version. Subsequently, the generated code is debugged to ensure it is free from errors, resulting in the extraction of a clean code version referred to as the Gold Standard. This Gold Standard serves as a benchmark for accuracy and effectiveness. To enhance future translation tasks, the Gold Standard is integrated into the translator's training regime. The clean code version is further refined into an executable programming script, which is then stored locally for future use. The process concludes with the final storage of the executable programming script.

The novelty of this framework lies in its ability to fully automate the translation process while maintaining accuracy and consistency through rigorous comparison and refinement steps. The integration of user interaction modules, which facilitate input requests, chat communication, code saving, and the storage of examples and gold standards, enhances the framework's usability. This makes the task more accessible and user-friendly, particularly for non-experts.

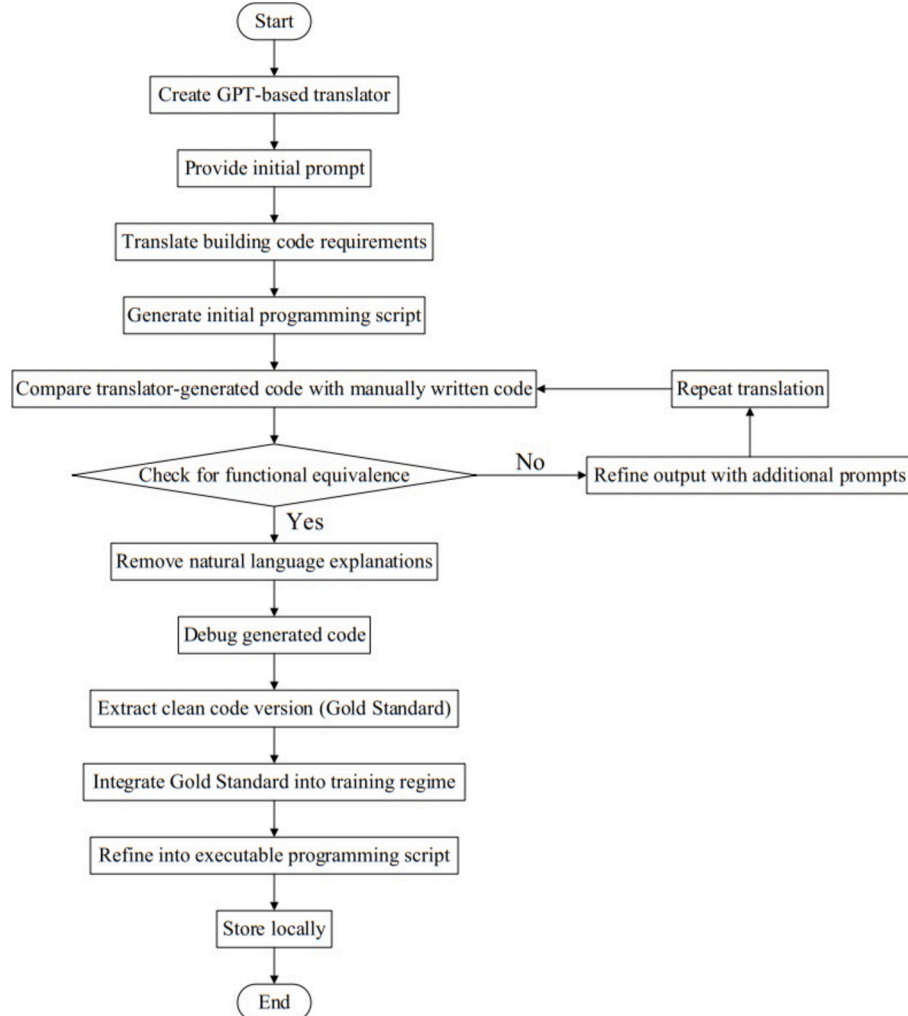


Fig. 3. UML activity diagram of the process flow.

3.4. Evaluation metrics

The assessment of the proposed methodology encompasses three critical dimensions: **effectiveness, versatility, and efficiency**. The determination of effectiveness hinges on addressing two pivotal inquiries: 1) the degree of functional deviation between the generated programming script and its manually written counterpart, and 2) the discrepancy between the functionally equivalent generated script and the gold standard. The resolution of the first inquiry ascertains the generated script's fidelity in representing building code requirements, whereas the second ensures the generated code's executability. Given the translator's performance variability, attributable to factors such as prompt misinterpretation and inconsistencies in parsing building regulation text, quantitatively gauging the functional disparity between the generated and manually written scripts poses challenges (although this variability tends to diminish as the experiment progresses). To approximate the gap for the first inquiry, the authors record the number of additional prompt rounds required by the translator in each experiment. For the second inquiry, three distance-based metrics (Levenshtein Distance, Jaro-Winkler Similarity, and Jaccard Distance) along with classic evaluation indicators (precision, recall, and the F1 score) are employed to compare the script generated (without natural language explanations) against the gold standard. The rationale for selecting these metrics is to provide a multi-dimensional assessment of the generated programming script's accuracy and similarity to the gold standard. Levenshtein Distance captures fine-grained textual differences, Jaro-Winkler Similarity measures character-level similarity with an emphasis on prefix matching, and Jaccard Distance evaluates the overall content overlap. Precision, recall, and the F1 score are classic indicators used to assess the script's correctness, completeness, and balance between precision and recall, respectively. These metrics are chosen to ensure robust evaluation and consistency with previous research methodologies [54]. The exclusion of the translator's directly generated programming script from this comparison is twofold: firstly, the embedded natural language (explanations and descriptions) complicates and potentially compromises the measurement's accuracy; secondly, assessing proximity to the gold standard is meaningless without first verifying functional congruence with manually written code.

To facilitate a nuanced comparison of code similarity across varying levels of detail, the authors compare the generated code without natural language explanations and the gold standard at the three levels of logic clause, predicate, and predicate-element, using the forementioned evaluation indicators. A logic clause may be either a logic fact or a logic rule, predicates are the basis of both logic facts and logic rules, and the predicate-element level dissects a predicate into its constituent elements. For instance, the predicate "building("OfficeBuilding", height, 70)." comprised of four distinct elements. Thus, the levels of detail are ordered from the most general to the most specific: logic clause level, predicate level, and predicate-element level. Analyzing these three granularity levels offers a comprehensive method for comparing the similarity between the generated script without natural language explanations, and the gold standard.

In evaluating the approach's robustness, the methodology is further applied to the translation of crash reports to ascertain its efficacy with a different source of natural language data. Furthermore, the code generation time during these crash report experiments is meticulously documented. This test serves to facilitate a comparative analysis with prior studies and assess the efficiency of the approach.

4. Experiment

4.1. Experimental plan

The experimental procedures for both the building code translation and crash report translation were meticulously designed to evaluate the proposed prompt-based framework. The workflow for both experiments

follows a systematic approach to ensure consistency, accuracy, and reliability in the translation process. Fig. 4 shows the general procedure for both experiments, which involves six main steps:

1. **Initialization:** set up the GPT-based translator with foundational information.
2. **Translation:** provide the input text (building code requirements or crash report descriptions) to the translator along with specific prompts to generate the initial programming script.
3. **Validation:**
 - perform a functional check by comparing the generated script with a manually written counterpart.
 - refine the output with additional prompts if discrepancies are found until functional equivalence is achieved.
4. **Processing:**
 - remove natural language explanations from the script using regular expression-based algorithms.
 - conduct a grammatical check to debug and refine the script into the gold standard.
5. **Storage:**
 - save the requirement/report and its corresponding gold standard as key-value pairs.
 - store the final programming script as Picat files.
6. **Execution:**
 - execute the script within the Picat programming environment.
 - evaluate the output based on predefined criteria.

To provide an understanding of the experimental setup and evaluation criteria, Table 2 presents an overview of specific changes in parameters and approaches for each experiment. The details of the procedures for each experiment are illustrated in Sections 4.3 and 4.4. Table 3 provides the length characteristics of the two datasets. The building code translation dataset contains longer texts, with an average length of 162.9 characters compared to 98.2 characters in the crash report translation dataset. The standard deviation indicates more variability in text length within the building code translation dataset (49.82) compared to the crash report translation dataset (24.84). Overall, the building code translation dataset is characterized by longer and more variable text entries, likely reflecting the detailed and complex nature of building code descriptions. In contrast, the crash report translation dataset consists of shorter and more consistent text entries, possibly due to the more straightforward and flexible nature of crash narratives. Both experiments were conducted using a laptop with an 11th Gen Intel Core i7-1185G7 central processing unit (CPU) and 16.0 GB of random-access memory (RAM).

4.2. Create GPT-based translator

To implement the proposed framework, a user interface platform based on an executable file is developed, as depicted in Fig. 5. This platform comprises three primary components: a chat window, a user entry field, and function buttons, each designed to fulfill specific operational roles. The chat window serves both to display responses from the backend module and to display the chat history. The user entry field allows for the input of tasks and prompts by users. The function buttons are engineered to execute various tasks. In particular, the "Save Picat Code" function processes the generated code into the final programming script. The "Background Information" function provides access to foundational knowledge about the target logic programming language, Picat, including its fundamental grammar and reasoning mechanisms, as illustrated in Fig. 6. The "Example Rules" function is designated for displaying and storing example rules and the gold standard, with Fig. 7 detailing the information of the initial example rule integrated into the



Fig. 4. General procedure for experiments.

Table 2

Comparison of parameters and approaches for each experiment.

Parameter/ Experiments	Building code translation	Crash report translation
Initial translation model	GPT-based translator	Refined translator model after building code translation
Dataset	51 quantitative fire protection system requirements from IBC 2015 [44].	20 quantitative descriptions from 2022 Michigan state traffic crash data.
Input dataset nature	Formal regulatory language	Narrative natural language
Example rule	"An automatic sprinkler system shall be provided where multiple fire areas of Group A-1, A-2, A-3 or A-4 occupancies share exit or exit access components and the combined occupant load of these fire areas is 300 or more." [44]	"Unit one driver stated he was traveling at 45–50 miles per hour, due to inclement weather."
Domain specificity	Fire protection systems, building codes	Vehicular accidents, crash reports
Quantitative focus	Occupant load, sprinkler system requirements, spatial distance, etc.	Speed, distance, time, breath test results, etc.
Initial prompt	"Based on the example you have, can you convert the following building code clause into logic facts and rules? Please concentrate on quantitative requirements and adhere to the provided format. Here is the building code clause: ..."	"Can you translate the following crash report into logic programming code? Please concentrate on quantitative information. Here is the crash report description: ..."
Output	Initial programming script with logic facts and logic rules.	Initial programming script encapsulating factual elements of the accident and logic rules for compliance.
Functional check	Compare generated script against a manually written counterpart which represents the specific requirement.	Ensure rules accurately filter out facts that conform to crash report descriptions.
Refinement process	Additional prompts to address discrepancies	Additional prompts to address discrepancies
Grammatical check	Debug and refine into the gold standard	Debug and refine into the gold standard
Storage method	Save as key-value pairs in the example rules	Save as key-value pairs in the example rules
Final script storage	Picat files	Picat files
Execution environment	Picat programming environment	Picat programming environment
Evaluation metrics	Functional equivalence, accuracy	Functional equivalence, accuracy
Total output files	100 Picat code files (2 per requirement: gold standard and functionally equivalent code)	40 Picat code files (2 per crash report narrative: gold standard and functionally equivalent code)

Table 3

Comparison of length information between two experimental datasets.

Dataset length information	Building code translation	Crash report translation
Min length	76	47
Average length	162.9	98.2
Max length	272	144
Standard deviation	49.82	24.84
Confidence intervals (95 %)	[147.74, 177.06]	[86.57, 109.83]

model. This initial rule demonstrates to the translator the method of analyzing building code requirements and translating them into specific logic language expressions. Alongside the initial example rule, gold standards developed throughout the experiments are also archived here as example rules, all organized within a Python dictionary (version 3.9.10) [73] with building code requirements as keys and their corresponding logic programming code as values. This accumulating repository of example rules enriches the database, progressively enhancing the model's performance. Moreover, "Built-in Functions" store built-in functions that aid in unit comparisons and conversions as well as time recording, as showcased in Fig. 8. These functions aim to rectify unit inconsistencies in building regulations and design information, thereby facilitating the compliance checking process. This user interface platform simplifies interaction with the translator, enabling users to review, process, and store the final programming script efficiently.

The efficacy and adaptability of the proposed framework were evaluated through two distinct experiments. To assess efficacy, fifty-one specific building code requirements from Chapter 9 of the International Building Code 2015 (IBC 2015) [44] were selected for automatic translation into Picat logic programs to support logic reasoning and compliance verification. Chapter 9 of IBC 2015 covers the requirements for building fire protection systems and includes 16 sections detailing automatic sprinkler systems, standpipe systems, smoke control systems, etc. The provisions related to quantitative requirements were chosen for this experiment because: 1) these clauses have clear logical relationships that facilitate the implementation of automatic reasoning, and 2) the quantitative requirements provide essential guidance for space configuration in buildings and building fire protection systems. An example of a provision used as a test case is "openings shall have a minimum dimension of not less than 30 inches" (Provision 903.2.11.1.1) [44], which included "30 in." as the main quantitative requirements for the dimensions of openings.

For the adaptability assessment, the platform was applied to the translation of twenty crash narratives from the 2022 Michigan state traffic crash data [45] into Picat logic programming language, illustrating the methodology's versatility. The crash narratives were manually selected, following the same criteria used for the selection of building code provisions, to determine if the translator could effectively grasp the logical relationships and quantitative descriptions present in another natural language document (i.e., crash reports). The quantitative descriptions of crash accidents are fundamentally different from those in building codes. Quantitative information from crash reports included time, speed, preliminary breath test (PBT) results, etc. For example, "unit one driver stated he was traveling at 45–50 miles per hour, due to inclement weather", included the speed "45–50 miles per hour" as the main quantitative information. Further details of these experiments are discussed in subsequent sections.

4.3. Building code translation

For this experiment, the authors selected fifty-one quantitative fire protection system requirements from the IBC 2015, aligning with prior research efforts [8]. In the experiment, one requirement was designated as an example rule (training data), whereas the remaining requirements served as test cases. The selection of the training data for this experiment was meticulously guided by several key criteria to ensure the effectiveness of the chosen example rule in training the GPT-based translator. First, the complexity of the example rule was a crucial factor. The rule needed to embody complex concepts and logical relationships to serve as a challenging yet instructive training example for the GPT model. By

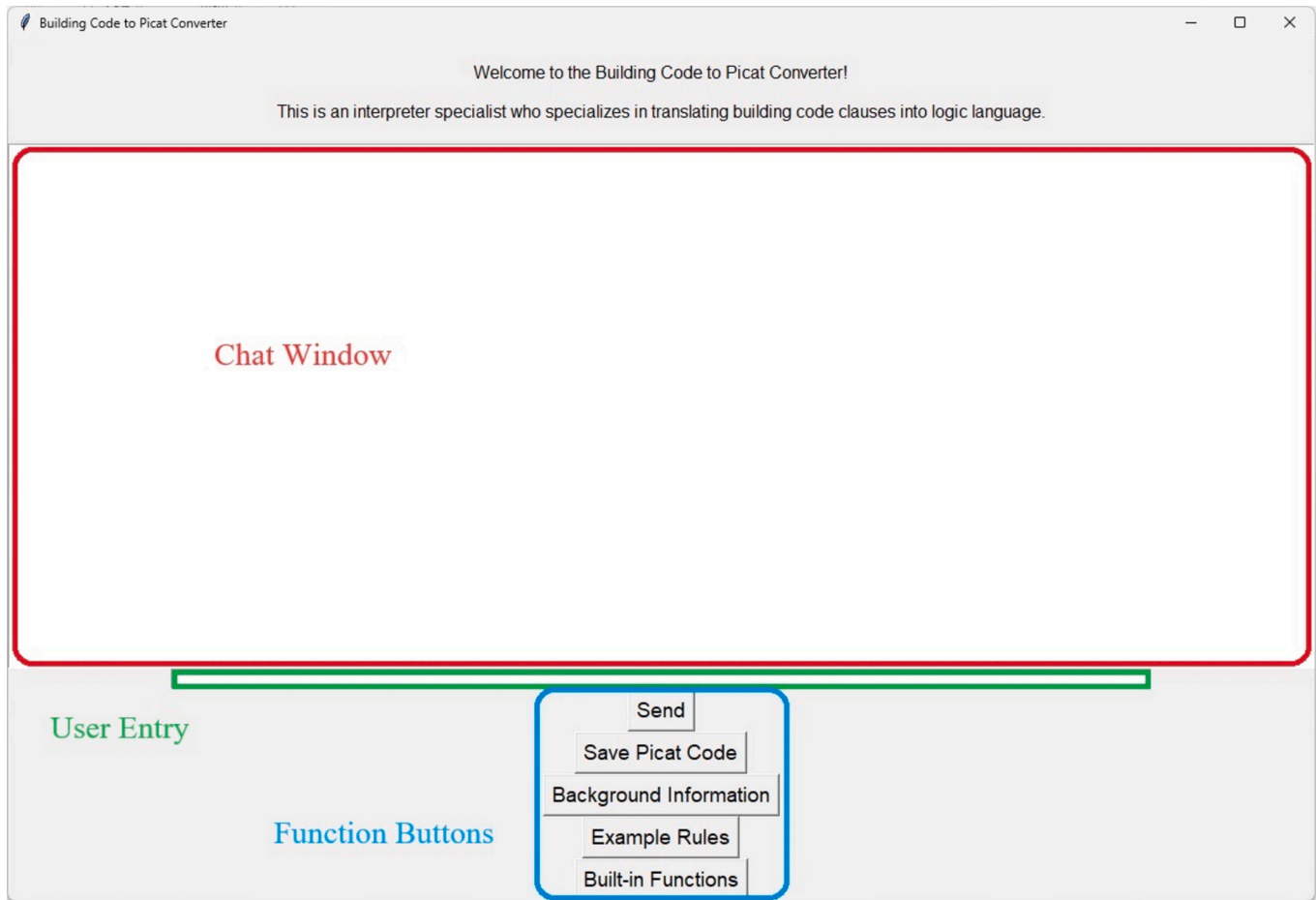


Fig. 5. User interface.

including such complexity, the example rule could effectively train the model to understand and represent intricate building code requirements. Second, the inclusion of quantitative requirements was essential. The example rule had to contain specific quantitative elements to align with the experiment's focus on translating quantitative fire protection system requirements. This alignment ensured that the model was trained to handle numerical data accurately within the context of logic programming. Lastly, the illustrative value of the example rule was a significant criterion. The rule was chosen for its ability to clearly illustrate the application of logic facts and logic rules, making it an ideal example for the model to learn from. By providing a clear and instructive example, the selected rule facilitated the model's learning process, enhancing its ability to translate similar building code requirements effectively.

The example rule: "An automatic sprinkler system shall be provided where multiple fire areas of Group A-1, A-2, A-3 or A-4 occupancies share exit or exit access components and the combined occupant load of these fire areas is 300 or more." [44], illustrating the requirement for automatic sprinkler systems in certain occupancy groups with a combined occupant load of 300 or more, was accompanied by corresponding logic facts and logic rules as depicted in Fig. 7. The logic rules emphasize the assessment of occupant load to determine the necessity for sprinkler system installation, categorizing scenarios into compliant or non-compliant based on the occupant load.

Upon initializing the translator, a new building code requirement is provided as input. The translator is then prompted with Question (1):

"Based on the example you have, can you convert the following building code clause into logic facts and rules? Please concentrate on quantitative

requirements and adhere to the provided format. Here is the building code clause: ..."

The prompts used in the experiments were designed to mimic natural language as used by non-experts, aiming to make the translator user-friendly. The key considerations behind the design included: 1) emphasizing specific numerical values or measurements within the building code clauses, 2) using simple and clear language to avoid ambiguity, and 3) maintaining a consistent structure across all prompts to facilitate comprehension and processing by the translator. An example of a successful prompt follows the form of "Question (1) + a specific building code requirement," such as:

"Based on the example you have, can you convert the following building code clause into logic facts and rules? Please concentrate on quantitative requirements and adhere to the provided format. Here is the building code clause: 'Openings shall have a minimum dimension of not less than 30 inches.'"

Unsuccessful prompts were generally ambiguous or introduced new concepts, causing the translator to be confused or misunderstand the target, or resulting in outputs that did not follow the given format. For example:

"Convert this building code into logic rules: 'Openings shall have a minimum dimension of not less than 30 inches.'"

This prompt does not specify the focus of scope (i.e., quantitative requirements) or clearly state the format to be followed. Due to the creativity of the GPT model, the output of the translator is not satisfactory.

Figure 9 shows a screenshot of the dialog interface in the experiment.

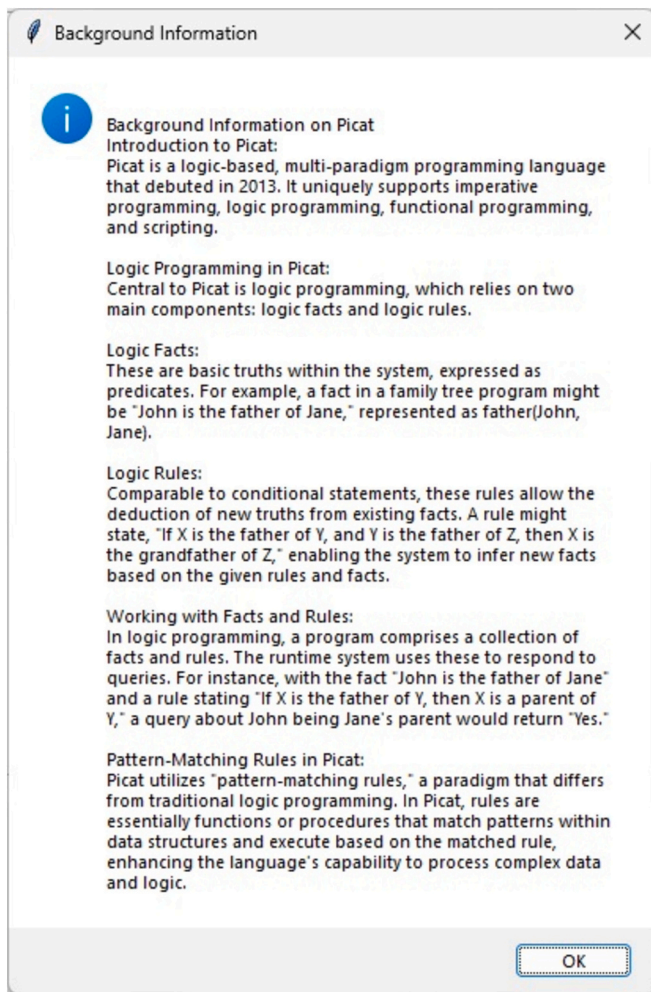


Fig. 6. Background information.

Following this, the generated programming script is subjected to functional and grammatical checks to ensure its readiness as an executable program. The functional check compares the generated script against a manually written counterpart for functional equivalence, necessitating further prompts for error correction until functional equivalence is achieved. Subsequent to functional validation, the script, typically

containing natural language explanations such as “Here is the generated code: ...”, undergoes processing via regular expression-based algorithms to extract the code. This code then undergoes a grammatical check for debugging and refinement into the gold standard.

By utilizing the “Save Picat Code” function, users re-enter the building code requirement, facilitating the storage of the requirement and its corresponding gold standard as key-value pairs in the example rules, as shown in [Figs. 10 and 11](#). Consequently, the size of the example rule set increases as the experiment progresses. These updated example rules serve to progressively enhance the model’s performance for future tasks. Additional functions are automatically integrated into the gold standard, guiding users to save the final programming script as a Picat file on their computer, demonstrated in [Fig. 12](#). Users can then execute this script within the Picat programming environment, with [Figs. 13 and 14](#) showcasing a script and its execution outcomes. In this example, the translator formulated logic rules for a building code requirement, generating three instances for logic reasoning, where two met the compliance criteria, and one did not. Throughout the experiment, each of the fifty test building code requirements was processed in this manner, yielding a total of one hundred Picat code files for evaluation, with two files generated for each requirement: a gold standard and a functionally equivalent code.

4.4. Crash report translation

Following the building code translation experiments, the refined translator model was further employed to convert crash reports into logic programming scripts without any adaptations on the structure of the model. These reports, compiled by police officers based on witness testimonies from automobile accidents, present detailed information in natural language. The transformation of these narratives into machine-readable formats, such as logic programming scripts, is advantageous for reconstructing vehicular accidents. This reconstruction is pivotal in supporting the safety evaluation of autonomous vehicles. Given the distinct linguistic characteristics of crash reports compared to building codes, these reports serve as an appropriate medium to assess the model's versatility.

For this experiment, twenty crash report narratives containing quantitative data on variables such as speed, distance, time, and preliminary breath test (PBT) results were manually selected as test cases. Each data was introduced to the model with a specific prompt (Question 2):

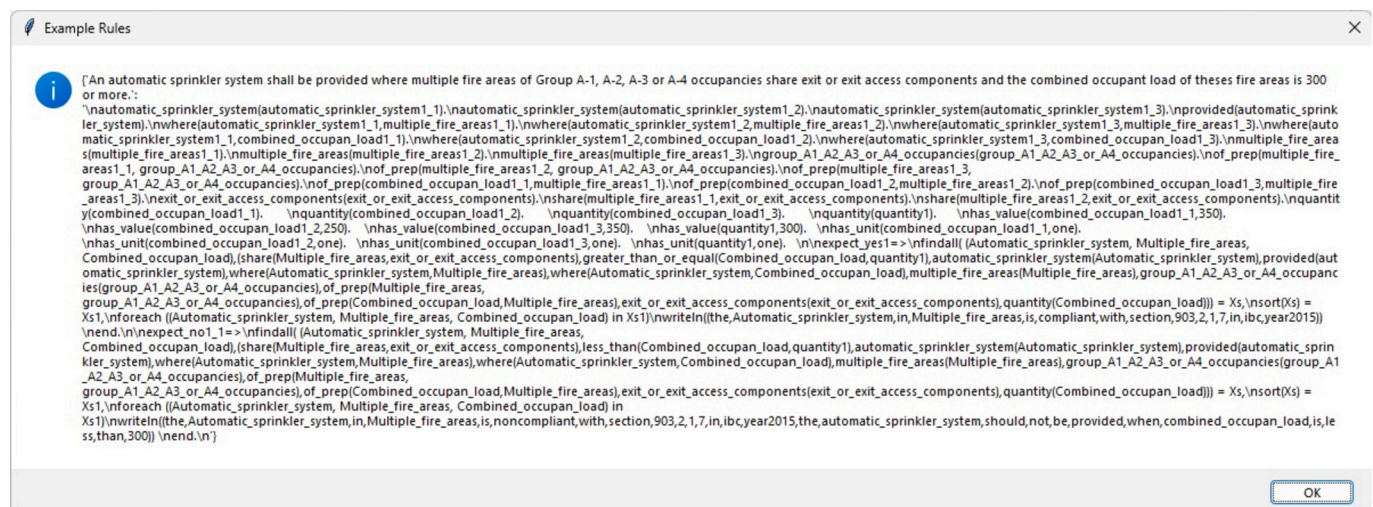


Fig. 7. Example rule.



```

i
greater_than_or_equal(A,B)=>
  quantity(A), quantity(B), has_value(A,V1), has_value(B,V2),
  has_unit(A,U1), has_unit(B, U2),
  if U1==U2
  then V1>=V2
  else convert_quantity(V1,U1,U2,V3),V3>=V2
  end.

less_than(A,B)=>
  quantity(A), quantity(B), has_value(A,V1), has_value(B,V2),
  has_unit(A,U1), has_unit(B, U2),
  if U1==U2
  then V1<V2
  else convert_quantity(V1,U1,U2,V3),V3<V2
  end.

less_than_or_equal(A,B)=>
  quantity(A), quantity(B), has_value(A,V1), has_value(B,V2),
  has_unit(A,U1), has_unit(B, U2),
  if U1==U2
  then V1<=V2
  else convert_quantity(V1,U1,U2,V3),V3<=V2
  end.

greater_than(A,B)=>
  quantity(A), quantity(B), has_value(A,V1), has_value(B,V2),
  has_unit(A,U1), has_unit(B, U2),
  if U1==U2
  then V1>V2
  else convert_quantity(V1,U1,U2,V3),V3>V2
  end.

convert_quantity(V1,U1,U2,V3)=>
if V1==0 then V3=0
elseif ((U1==feet;U1==foot), (U2==inches;U2==inch)) then V3
= V1*12
elseif ((U1==inches;U1==inch), (U2==feet;U2==foot)) then V3
= V1/12
elseif ((U1==feet,U2==foot);(U1==foot,U2==feet);
(U1==inches,U2==inch);(U1==inch,U2==inches);
(U1==foot_candles,U2==foot_candle);(U1==foot_candle,U2==
foot_candles);
(U1==story,U2==stories); (U2==story,U1==stories);
(U1==story,U2==storey);(U2==story,U1==storey);
(U1==square_feet,U2==square_foot);
(U2==square_feet,U1==square_foot);
(U1==square_inch,U2==square_inches);(U2==square_inch,U1
==square_inches);
(U1==story,U2==stories); (U2==story,U1==stories);
(U1==meter,U2==meters); (U2==meter,U1==meters);
(U1==centimeter,U2==centimeters);(U2==centimeter,U1==ce
ntimeters)
) then V3 = V1*1
elseif (U1==square_feet,U2==square_inches) then V3 =
V1*144
elseif (U2==square_feet,U1==square_inches) then V3 =
V1/144
elseif (U1==meter,U2==centimeter) then V3 = V1*100
elseif (U2==meter,U1==centimeter) then V3 = V1/100
elseif (U1==centimeters,U2==millimeters) then V3 = V1*10
elseif (U2==centimeters,U1==millimeters) then V3 = V1/10
end.

check_time => statistics(runtime,T), writeIn(T).
  
```

Fig. 8. Built-in functions.

“Can you translate the following crash report into logic programming code? Please concentrate on quantitative information. Here is the crash report description: ...”

The translation process follows the framework applied in the building code translation, aiming to generate a final programming script for

each crash report narrative description. The translator was tasked with creating multiple logic facts to encapsulate the factual elements of the accidents and devising logic rules to ascertain compliance with the crash report narratives. The functional check of the generated programming script is based on whether the rule can accurately filter out the facts that conform to the description of the crash report. Figs. 15 and 16 illustrate an example of this translation process, highlighting a crash report narrative description alongside its resultant code. This particular example concentrated on verifying the compliance of simulated driver speeds with the speeds reported. Upon conclusion of this experiment, a total of forty Picat code files were compiled from the twenty crash report narrative descriptions for comprehensive evaluation.

5. Results and discussion

5.1. Effectiveness of the proposed framework

The effectiveness of the proposed framework was substantiated by quantifying the proximity of the generated code to a corresponding executable script that accurately encapsulates the building code requirements. This evaluation was conducted through two primary metrics: the quantity of additional prompts necessitated to attain functional equivalence with the manually written script, and the textual similarity between the generated code and the gold standard after reaching this equivalence. To facilitate this analysis, automated calculation methods were devised and implemented via Python code.

5.1.1. Text similarity measurements results

Upon submitting the initial task request through the prompt, the process meticulously records the number of additional prompt rounds necessary until the generated code achieves functional equivalence with the manually written script (Fig. 1), meticulously crafted to encapsulate the nuances of building code concepts and requirements comprehensively. Table 4 illustrates the distribution of these additional prompts across 50 test cases, revealing that the maximum number of prompt rounds required is four, with 32 cases achieving functional equivalence without necessitating any further prompts. This indicates a high accuracy of the translator in interpreting and converting building code information into programming script. A detailed number of further prompts needed in each experiment is shown in Fig. 18. The variation in the number of additional prompts is attributed primarily to the different complexities of the building code requirements. Through results analysis the main issues necessitating further prompts are identified as follows: 1) omission of key building code concepts or their relationships in logic facts, 2) direct comparison of quantities without employing built-in functions for numerical comparisons with unit conversions, 3) undefined predicates in logic rules, 4) failure to define at least two instances to evaluate logic rules in compliant and non-compliant scenarios, 5) absence of logic rules for identifying instances of non-compliance, 6) overlooking checks for quantitative requirements, and 7) logical inaccuracies, such as misinterpretation that noncompliance indicates the failure to meet all/both conditions when there are more than one conditions, such as the example conditions “*greater_than_or_equal(VestibuleArea, min_net_area_threshold)*” and “*greater_than_or_equal(OuterSpaceWidth, min_width_threshold)*”, where failing to meet one of them should be considered as noncompliance. These challenges predominantly stem from the inherent complexity of building codes, the task’s difficulty, and the translator’s variability in performance. However, these issues can be addressed with further prompts that clearly indicate errors in the model’s responses, suggest revisions, and request next steps. As the experiment progresses, these issues are gradually reduced. Overall, most translations can achieve functional equivalence within two rounds of prompts, which is considered acceptable in practice [46].

Upon reaching functional equivalence, the generated code, with comments in natural language (e.g., the second paragraph in the console

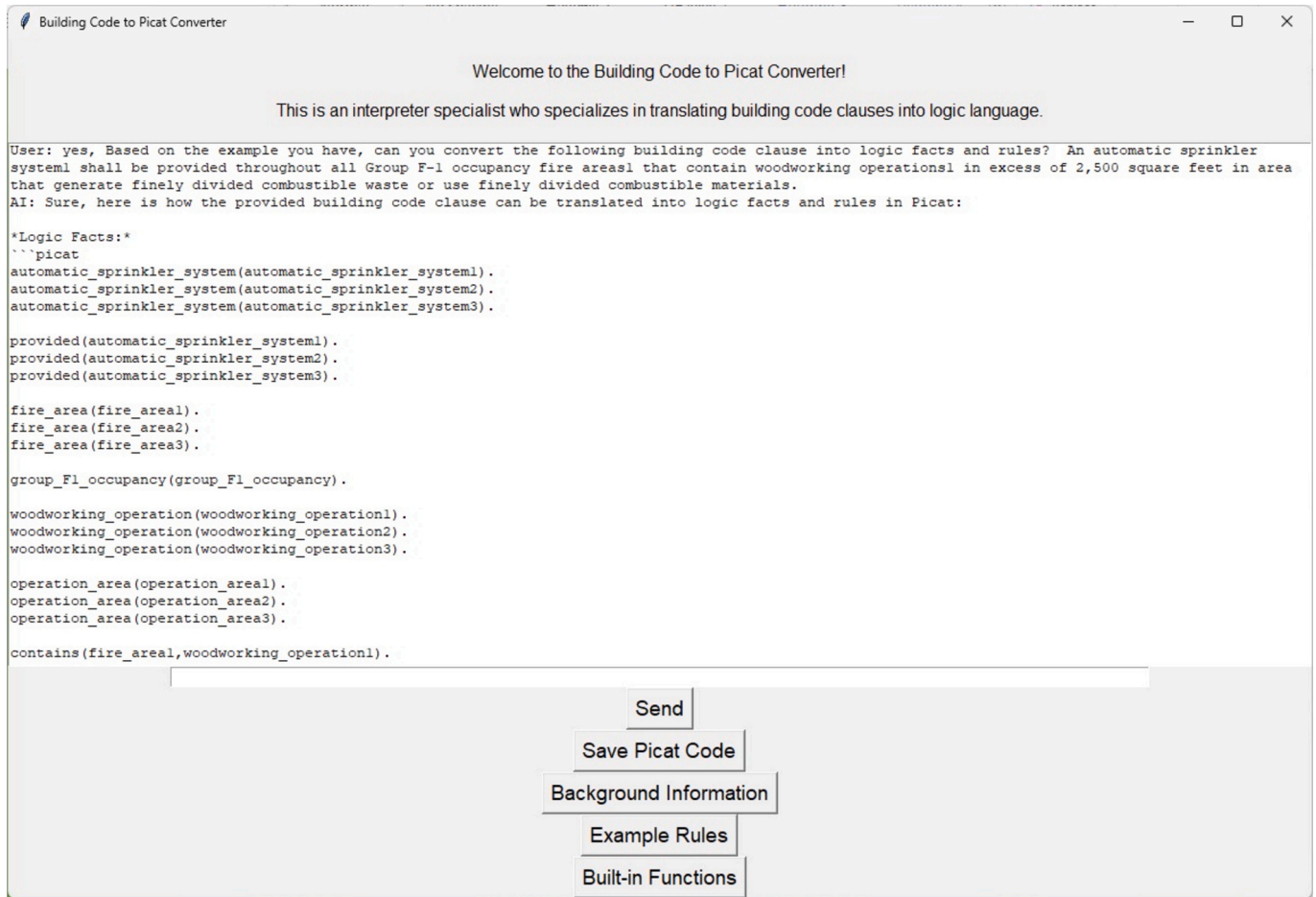


Fig. 9. Example of chat interface.

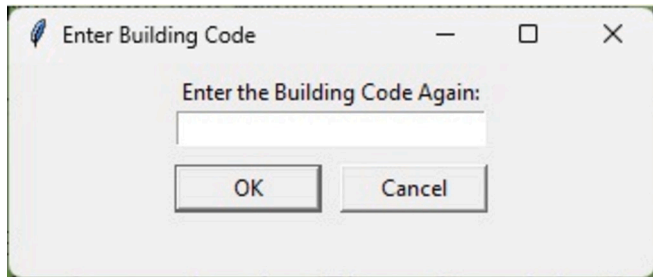


Fig. 10. Users are asked to re-enter the building code to store the gold standard. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

of Fig. 9) deleted, was evaluated against the gold standard. One main grammar issue in the generated code was that predicates sharing the same name are not defined consecutively, leading to execution errors on the Picat platform. **To quantitatively assess textual similarity between the generated code and the gold standard, metrics such as the Levenshtein Distance, Jaro-Winkler Similarity, and Jaccard Distance were employed.** Fig. 17 presents the distribution of Levenshtein Distances across all test cases. Given that the Levenshtein Distance measures the minimum number of single-character modifications needed to transform one string into another [54,85], its values range from zero to the length of the longer string, making statistical distribution analysis more meaningful than average values. Among the 50 test cases, 20 exhibit a Levenshtein Distance of zero, signifying perfect identity with the gold

standard and absence of grammatical errors. The highest observed Levenshtein Distance was 1237 edits, with 48 cases displaying distances under 1000 edits. It's important to note that the absolute Levenshtein distance should be considered in relation to the length of the codes being compared. For example, a distance of two might be considered small when comparing long words or phrases (e.g., "international" vs. "internationale"), but significant for short words (e.g., "cat" vs. "bat"). To address that, the Jaccard Distance, which shows relative insensitivity to code length, is utilized for further analysis.

The Jaccard Distance and Jaro-Winkler Similarity for 50 test cases were computed and are depicted in Fig. 18. These two metrics inversely correlate: a higher similarity yields a Jaccard Distance approaching 0 and a Jaro-Winkler Similarity nearing 1. The mean values for the Jaccard Distance and Jaro-Winkler Similarity across all tests were found to be 0.0100 and 0.9535, respectively. This high level of similarity between the generated code and the gold standard demonstrates their close textual alignment, transcending the consideration of character order (noting that Jaro-Winkler Similarity accounts for character order, whereas Jaccard Distance does not). Additionally, a notable consistency exists between the Jaccard Distance, Jaro-Winkler Similarity, and the number of additional prompts required during the experiments. As illustrated in Fig. 18, instances necessitating more prompts, indicative of challenges in accurately translating specific building code requirements, correlate well with larger Jaccard Distances and lower Jaro-Winkler Similarities.

Considering these four metrics, it is apparent that despite occasional requirements for additional prompts to address translation issues, the translator succeeds in generating codes that not only reflect the building code requirements accurately but also exhibit a high degree of textual

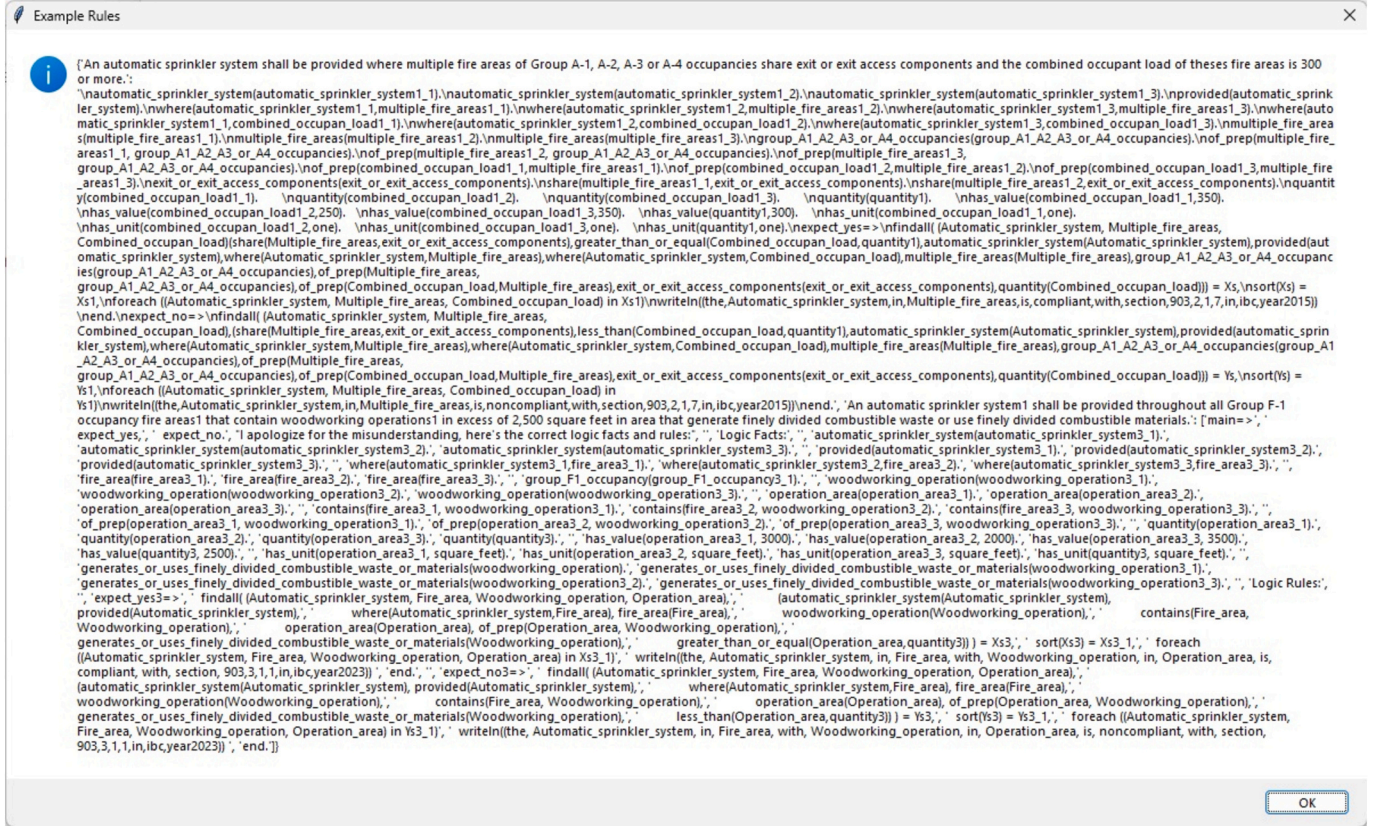


Fig. 11. Example of updated example rules.

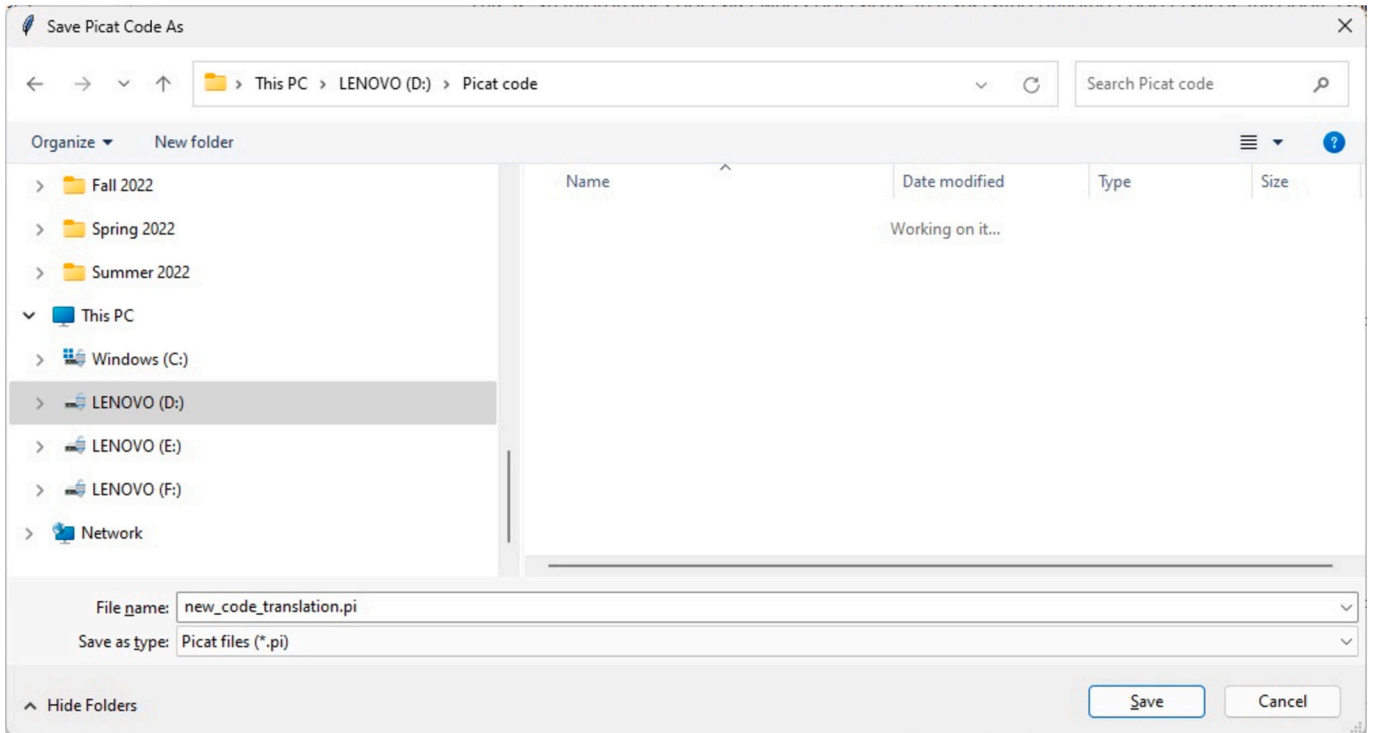


Fig. 12. Save final programming script.

similarity to the gold standards, within a reasonable number of prompts.

5.1.2. Precision, recall and F1 score at logic clause level, predicate level, and predicate-element level

To conduct a nuanced comparison between the generated code and

```

1  main=>
2      expect_yes3,
3      expect_no3.
4
5  automatic_sprinkler_system(automatic_sprinkler_system2_1).
6  automatic_sprinkler_system(automatic_sprinkler_system2_2).
7  automatic_sprinkler_system(automatic_sprinkler_system2_3).
8
9  provided(automatic_sprinkler_system2_1).
10 provided(automatic_sprinkler_system2_2).
11 provided(automatic_sprinkler_system2_3).
12
13 where(automatic_sprinkler_system2_1,fire_area2_1).
14 where(automatic_sprinkler_system2_2,fire_area2_2).
15 where(automatic_sprinkler_system2_3,fire_area2_3).
16
17 fire_area(fire_area2_1).
18 fire_area(fire_area2_2).
19 fire_area(fire_area2_3).
20
21 group_F1_occupancy(group_F1_occupancy).
22
23 woodworking_operation(woodworking_operation2_1).
24 woodworking_operation(woodworking_operation2_2).
25 woodworking_operation(woodworking_operation2_3).
26
27 operation_area(operation_area2_1).
28 operation_area(operation_area2_2).
29 operation_area(operation_area2_3).
30
31 contains(fire_area2_1, woodworking_operation2_1).

```

Fig. 13. Screenshot of final programming script of a building code requirement.

the gold standard, metrics such as precision, recall, and the F1 score were computed at three granularity levels: logic clause level, predicate level, and predicate-element level. An illustrative example below demonstrates the calculation of precision and recall at the logic clause level.

Table 5 presents a set of generated code with the gold standard and the associated building code requirement, highlighting the discrepancies between them. In the evaluation, each logic fact or rule constitutes a logic clause. For instance, the entry in the first row, “*design_area (design_area3_1).*”, is treated as a single logic fact in Picat, thus considered a single evaluation unit. Similarly, the entry in the 21st row introduces a Picat logic rule, identified by “*expect_yes3*”, also regarded as a singular unit for comparison purposes. The analysis of this particular instance reveals that the translator’s output closely aligns with the gold standard, barring a minor error related to the incorrect capitalization of “NOT”. Within the Picat syntax, capitalization denotes variables, indicating a need for adjustment. In this case, the gold standard contained 21 logic clauses, with the generated code accurately reflecting 20 of these clauses, resulting in precision and recall metrics of 95.24 % at the logic

clause level.

The analysis extends further by breaking down the code snippets into predicates and predicate elements to ascertain their precision and recall metrics. The outcomes of these evaluations across each experiment are depicted in Figs. 19 and 20. Given the hierarchical structure from the most general to the most specific—logic clause, predicate, and predicate-element—it is anticipated that precision and recall metrics would exhibit an upward trend with increasing granularity. Nonetheless, anomalies observed in Figs. 19 and 20 seem to challenge this expectation. This discrepancy arises primarily because precision and recall at the predicate and predicate-element levels are derived through automated Python scripts, whereas the assessment at the logic clause level involves manual calculation, incorporating considerations for cascading errors and overlooking repeated mistakes in certain instances. Figs. 19 and 20 illustrate that, following initial variability, the translator’s accuracy exhibits a trend towards stabilization and improvement after the 13th experiment. However, there are notable exceptions, such as in the 49th experiment, where the translator encountered difficulties in accurately defining and incorporating several essential concepts and relationships relevant to the building code requirements. Additional factors contributing to drops in precision and recall include the incorporation of unnecessary conditions in logic rules and inconsistent instance information. Figs. 19 and 20 also elucidate a strong correlation between precision and recall, jointly reflecting the translator’s performance across the experiments.

Table 6 presents a summary of performance of the model on building code translation task, demonstrating high accuracy, consistency, and reliability across various similarity, precision, and recall metrics. The performance of the model on the building code translation task is exemplified by its high Jaro-Winkler Similarity and low Jaccard Distance. With a Jaro-Winkler Similarity mean score of 0.9535 and a low standard deviation of 0.0481, the model consistently produces translations that are highly similar to the gold standard. This high similarity is further reinforced by the 95 % confidence interval [0.9398, 0.9672], indicating reliable performance. Simultaneously, the model exhibits minimal dissimilarity, as evidenced by the Jaccard Distance mean score of 0.0100 and a standard deviation of 0.0192. The 95 % confidence interval [0.0046, 0.0155] supports the observation of low dissimilarity, highlighting the model’s ability to preserve the semantic content of the original texts. Together, these metrics demonstrate the model’s effectiveness in achieving high fidelity and semantic accuracy in building code translations.

The model’s performance on the building code translation task is impressive across all three levels of evaluation: logic clause level, predicate level, and predicate element level. Consistent with theoretical predictions, an increase in granularity leads to a gradual improvement in average precision and recall. At the logic clause level, the model achieves high precision, recall and F1 score, with mean scores of 0.9737, 0.9588 and 0.9662, respectively, indicating accurate and comprehensive capture of logic components. At the predicate level, the model maintains strong performance with mean precision of 0.9704, recall of

```

Picat 2.6#2, (C) picat-lang.org, 2013-2019.
Type 'help' for help.
Picat> cl(new_code_translation)
Compiling:: new_code_translation.pi
new_code_translation.pi compiled in 4 milliseconds
loading...

yes

Picat> main
the,automatic_sprinkler_system2_1,in,fire_area2_1,with,woodworking_operation2_1,in,operation_area2_1,is,compliant,with,ibc,year2015
the,automatic_sprinkler_system2_3,in,fire_area2_3,with,woodworking_operation2_3,in,operation_area2_3,is,compliant,with,ibc,year2015
the,automatic_sprinkler_system2_2,in,fire_area2_2,with,woodworking_operation2_2,in,operation_area2_2,is,noncompliant,with,ibc,year2015

yes

Picat> |

```

Fig. 14. Example of final programming script execution result.

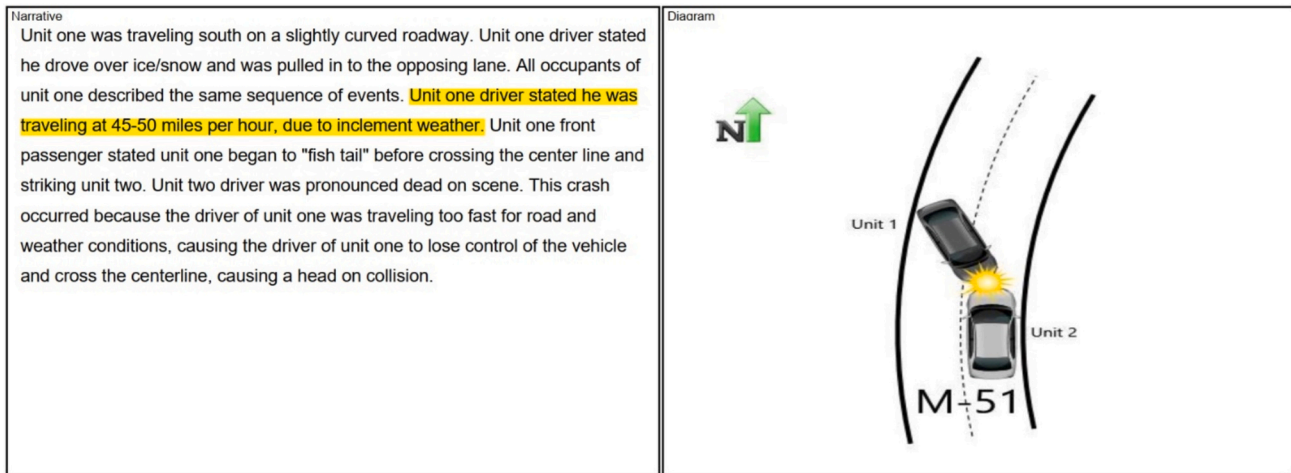


Fig. 15. Example crash report narrative.

```

34  expect_yes1 =>
35      findall(Driver,
36          (
37              entity(Driver),
38              has_reported_speed(Driver, ReportedSpeed),
39              has_value(ReportedSpeed, Speed),
40              has_unit(ReportedSpeed, mph),
41              greater_than_or_equal(Speed, lower_bound_speed),
42              less_than_or_equal(Speed, upper_bound_speed)
43          )
44      ) = CompliantDrivers,
45      sort(CompliantDrivers) = SortedCompliantDrivers,
46      writeln('Compliant drivers:'),
47      foreach(Driver in SortedCompliantDrivers)
48      | | writeln(Driver)
49      end.
50
51  expect_no1 =>
52      findall(Driver,
53          (
54              entity(Driver),
55              has_reported_speed(Driver, ReportedSpeed),
56              has_value(ReportedSpeed, Speed),
57              has_unit(ReportedSpeed, mph),
58              (
59                  less_than(Speed, lower_bound_speed);
60                  greater_than(Speed, upper_bound_speed)
61              )
62          )
63      ) = NonCompliantDrivers,
64      sort(NonCompliantDrivers) = SortedNonCompliantDrivers,
65      writeln('Non-compliant drivers:'),

```

Fig. 16. Screenshot of generated programming script for a crash report.

0.9632 and F1 score of 0.9668, showcasing its reliability in translating predicates accurately. Similarly, at the predicate element level, the highest precision, recall and F1 score are observed, with mean scores of 0.9779, 0.9701 and 0.9740, respectively, reflecting high accuracy and consistency in handling detailed elements of predicates.

When compared with state-of-the-art rule-based and machine

learning-based approaches in regulatory information extraction, as shown in Table 7, the translator exhibits superior performance at the logic clause level, underscoring its efficacy in accurately translating building code requirements into logic programming constructs. Table 8 presents a detailed example comparison between a rule-based approach [8] and the proposed method. When the two approaches are applied to

Table 4

Distribution of the number of further prompts needed in building code translation.

Number of further prompts	0	1	2	3	4
Count	32	7	9	1	1

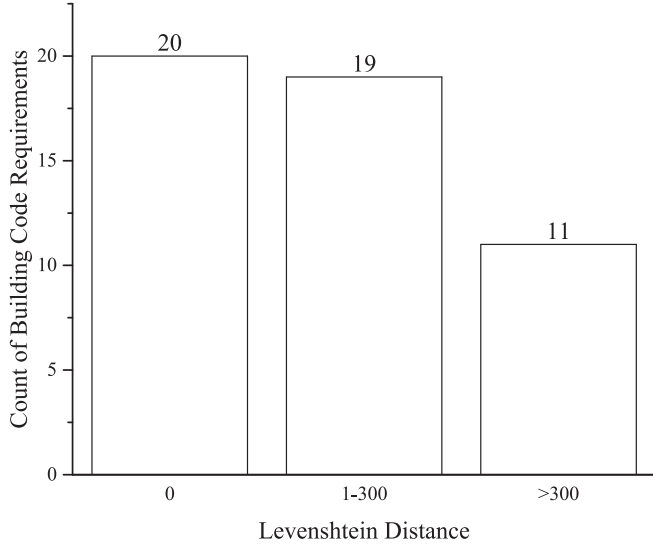


Fig. 17. Distribution of Levenshtein Distance in building code translation experiments.

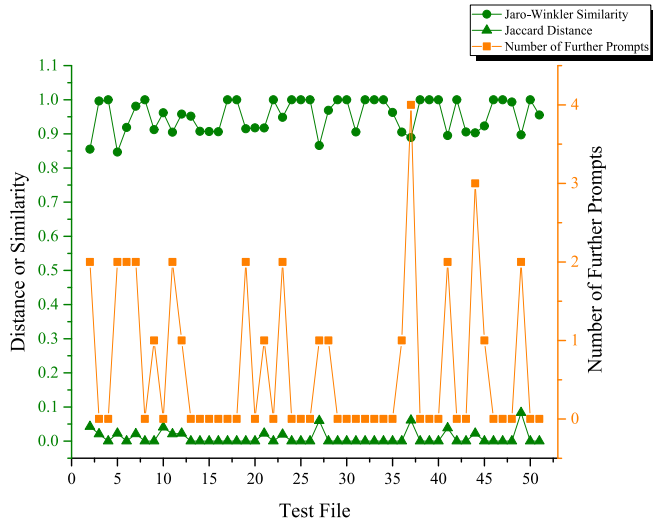


Fig. 18. Jaro Winkler Similarity, Jaccard Distance, and number of further prompts in building code translation experiments.

the same building code requirement, the main differences in the generated code are evident: The rule-based approach generated one logic rule for checking without generating logic facts and missed the building concept “any plan dimension”. The proposed method generated both logic facts and two logic rules for checking. After two rounds of further prompts, it created three entities for testing three scenarios and two logic rules for finding compliant and noncompliant entities. The generated code can be run directly in the Picat programming environment with built-in functions. Clearly, compared to the rule-based approach, the proposed method generates more comprehensive and complete inspection rules without requiring excessive manual efforts. This demonstrates the method’s potential to significantly enhance the

Table 5

Example pair of generated code and gold standard in the experiment.

Building code requirement	Generated code	Gold standard
“Where the design area of the sprinkler system consists of a corridor protected by one row of sprinklers, the maximum number of sprinklers required to be calculated is 13.” [44]	<pre> design_area(design_area3_1). design_area(design_area3_2). design_area(design_area3_3). consists_of(design_area3_1, corridor3_1). consists_of(design_area3_2, corridor3_2). consists_of(design_area3_3, corridor3_3). corridor(corridor3_1). corridor(corridor3_2). corridor(corridor3_3). row_of_sprinklers(row3_1). row_of_sprinklers(row3_2). row_of_sprinklers(row3_3). protected_by(corridor3_1, row3_1). protected_by(corridor3_2, row3_2). protected_by(corridor3_3, row3_3). number_of_sprinklers(row3_1, 14). number_of_sprinklers(row3_2, 12). number_of_sprinklers(row3_3, 13). max_allowed_sprinklers(13). </pre>	<pre> design_area(design_area3_1). design_area(design_area3_2). design_area(design_area3_3). consists_of(design_area3_1, corridor3_1). consists_of(design_area3_2, corridor3_2). consists_of(design_area3_3, corridor3_3). corridor(corridor3_1). corridor(corridor3_2). corridor(corridor3_3). row_of_sprinklers(row3_1). row_of_sprinklers(row3_2). row_of_sprinklers(row3_3). protected_by(corridor3_1, row3_1). protected_by(corridor3_2, row3_2). protected_by(corridor3_3, row3_3). number_of_sprinklers(row3_1, 14). number_of_sprinklers(row3_2, 12). number_of_sprinklers(row3_3, 13). max_allowed_sprinklers(13). </pre>
expect_yes3=>	<pre> findall((DesignArea, Corridor, RowOfSprinklers, Number), (design_area(DesignArea), consists_of(DesignArea, Corridor), corridor(Corridor), protected_by(Corridor, RowOfSprinklers), row_of_sprinklers (RowOfSprinklers), number_of_sprinklers (RowOfSprinklers, Number), max_allowed_sprinklers(Max), Number=<=Max))=Xs3, sort(Xs3)=Xs3_sorted, foreach ((DesignArea, Corridor, RowOfSprinklers, Number) in Xs3_sorted) writeln(the, DesignArea, with, Corridor, protected, by, RowOfSprinklers, with, Number, sprinklers, is, compliant, with, section, 903,4,2,in,ibc,year2023)) end. </pre>	<pre> expect_yes3=> findall((DesignArea, Corridor, RowOfSprinklers, Number), (design_area(DesignArea), consists_of(DesignArea, Corridor), corridor(Corridor), protected_by(Corridor, RowOfSprinklers), row_of_sprinklers (RowOfSprinklers), number_of_sprinklers (RowOfSprinklers, Number), max_allowed_sprinklers(Max), Number=<=Max))=Xs3, sort(Xs3)=Xs3_sorted, foreach ((DesignArea, Corridor, RowOfSprinklers, Number) in Xs3_sorted) writeln(the, DesignArea, with, Corridor, protected, by, RowOfSprinklers, with, Number, sprinklers, is, compliant, with, section, 903,4,2,in,ibc,year2023)) end. </pre>
expect_no3=>	<pre> findall((DesignArea, Corridor, RowOfSprinklers, Number), (design_area(DesignArea), consists_of(DesignArea, Corridor), corridor(Corridor), protected_by(Corridor, RowOfSprinklers), row_of_sprinklers (RowOfSprinklers), number_of_sprinklers (RowOfSprinklers, Number), max_allowed_sprinklers(Max), Number>Max))= Xs3_non_compliant, sort(Xs3_non_compliant)= </pre>	<pre> findall((DesignArea, Corridor, RowOfSprinklers, Number), (design_area(DesignArea), consists_of(DesignArea, Corridor), corridor(Corridor), protected_by(Corridor, RowOfSprinklers), row_of_sprinklers (RowOfSprinklers), number_of_sprinklers (RowOfSprinklers, Number), max_allowed_sprinklers(Max), Number>Max))= Xs3_non_compliant, sort(Xs3_non_compliant)= </pre>

(continued on next page)

Table 5 (continued)

Building code requirement	Generated code	Gold standard
	<i>Xs3_non_compliant_sorted, foreach ((DesignArea, Corridor, RowOfSprinklers, Number) in Xs3_non_compliant_sorted) writeln((the, DesignArea, with, Corridor, protected, by, RowOfSprinklers, with, Number, sprinklers, is, NOT, compliant, with, section, 903,4,2, in, ibc, year2023)) end.</i>	<i>Xs3_non_compliant_sorted, foreach ((DesignArea, Corridor, RowOfSprinklers, Number) in Xs3_non_compliant_sorted) writeln((the, DesignArea, with, Corridor, protected, by, RowOfSprinklers, with, Number, sprinklers, is, not, compliant, with, section, 903,4,2, in, ibc, year2023)) end.</i>

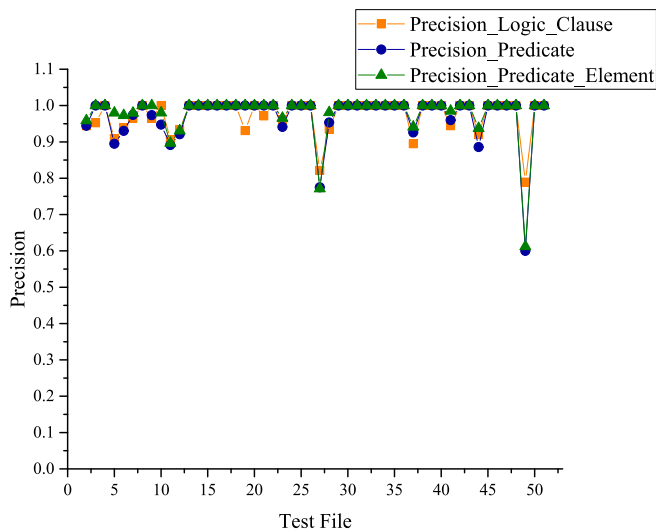


Fig. 19. Precision of building code translation experiments.

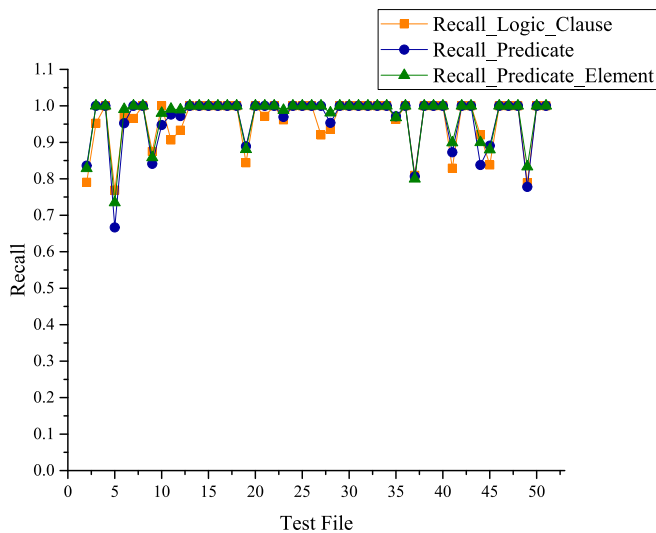


Fig. 20. Recall of building code translation experiments.

efficiency and accuracy of regulatory information transformation and compliance verification.

5.2. Versatility and efficiency of the proposed framework

The performance of the proposed framework on crash report

Table 6

Evaluation metrics for building code translation experiments.

Metrics	Mean	Standard deviation	Confidence intervals (95 %)
Jaro-Winkler similarity	0.9535	0.0481	[0.9398, 0.9672]
Jaccard distance	0.0100	0.0192	[0.0046, 0.0155]
Precision (logic clause)	0.9737	0.0466	[0.9605, 0.9869]
Recall (logic clause)	0.9588	0.0680	[0.9395, 0.9782]
Precision (predicate)	0.9704	0.0691	[0.9507, 0.9900]
Recall (predicate)	0.9632	0.0728	[0.9425, 0.9839]
Precision (predicate element)	0.9779	0.0649	[0.9594, 0.9963]
Recall (predicate element)	0.9701	0.0629	[0.9522, 0.9880]

Table 7

Performance comparison of proposed method with existing state-of-the-art methods.

	Precision	Recall	F1 score
The proposed approach in this study	97.4 %	95.9 %	96.6 %
Rule-based approach [8]	96.9 %	94.4 %	95.6 %
Machine learning-based approach [5]	93.1 %	92.9 %	93.0 %

translation was tested and analyzed to verify its adaptation. Similar to the verification of its effectiveness, the numbers of further prompts needed and text similarity between generated code without natural language explanations and golden standard were recorded and calculated. In addition, the time taken to generate the codes was recorded to assess its efficiency.

5.2.1. Text similarity measurements results

The translator after the building code translation task was used for crash report translation. The number of additional prompts needed across 20 cases was documented, with their distribution detailed in Table 9. Remarkably, 14 out of 20 translation instances required no further prompts to accurately represent the quantitative data within the crash reports, with the highest number of additional prompts capped at three. Fig. 22 provides an exhaustive breakdown of the additional prompts required for each case. Initial analyses identified the primary reasons for necessitating additional prompts: 1) failure of the translator to define a minimum of two instances for assessing compliance and non-compliance scenarios respectively, 2) omission in defining pertinent concepts or relationships from the crash reports, and 3) the inaccurate evaluation of instances based on prior knowledge (e.g., legal limits) instead of adhering to the specific details provided in the crash report description. These problems can be resolved by clearly identifying the errors in the model's answers, providing correction suggestions, and specifying the next steps in the further prompts. Over the course of the experiment, a notable decline in the requirement for further prompts was observed, culminating in a point after eight iterations where no additional prompts were needed for effective translation (Fig. 22).

Figure 21 displays the distribution of Levenshtein Distances across all evaluated cases, with the largest distance recorded at 171 edits, while the remainder falls below 100. Notably, 17 cases achieved a Levenshtein Distance of zero, indicating an exact match between the generated code and the gold standard. Furthermore, Fig. 22 presents the calculated Jaccard Distance and Jaro-Winkler Similarity. As the experiments advance, a significant enhancement in the translator's performance is observed. The overall average Jaccard Distance and Jaro-Winkler Similarity for all cases in the crash report translation experiment are 0.0022 and 0.9967, respectively, showing a high level of accuracy.

5.2.2. Precision, recall and F1 score at logic clause level, predicate level, and predicate-element level

Figures 23 and 24 present the precision, recall, and F1 score across

Table 8
Example comparison of rule-based approach and LLM-based approach.

Building code requirement	Generated code from rule-based approach [8]	Generated code from the proposed approach
"Habitable spaces, other than a kitchen, shall not be less than 7 ft in any plan dimension."		<pre>entity(habitable_space1). entity(habitable_space2). entity(habitable_space3). quantity (length_habitable_space1). quantity(width_habitable_space1). quantity (length_habitable_space2). quantity(width_habitable_space2). quantity (length_habitable_space3). quantity(width_habitable_space3). quantity (min_dimension_requirement). has_value (length_habitable_space1, 8). has_value(width_habitable_space1, 7). has_value (length_habitable_space2, 6). has_value(width_habitable_space2, 7). has_value (length_habitable_space3, 7). has_value(width_habitable_space3, 5). has_value (min_dimension_requirement, 7). has_unit(length_habitable_space1, feet). has_unit(width_habitable_space1, feet). has_unit(length_habitable_space2, feet). has_unit(width_habitable_space2, feet). has_unit(length_habitable_space3, feet). has_unit(width_habitable_space3, feet). has_unit (min_dimension_requirement, feet). has_length(habitable_space1, length_habitable_space1). has_length(habitable_space2, length_habitable_space2). has_length(habitable_space3, length_habitable_space3). has_width(habitable_space2, width_habitable_space2). has_width(habitable_space1, width_habitable_space1). has_width(habitable_space3, width_habitable_space3). dimension_compliance_check => findall(Space, (entity(Space), has_length(Space, Length), has_width(Space, Width), greater_than_or_equal(Length, min_dimension_requirement), greater_than_or_equal(Width, min_dimension_requirement))) = CompliantSpaces,</pre>
compliance_of_clear_passageway (Clear_passageway):- habitable_spaces (Habitable_spaces),kitchen (Kitchen),greater_than_or_equal (Any_plan_dimension,quantity (7,feet))		

Table 8 (continued)

Building code requirement	Generated code from rule-based approach [8]	Generated code from the proposed approach
		<pre>sort(CompliantSpaces) = SortedCompliantSpaces, writeln("Habitable spaces compliant with the building code requirement: "), foreach(Space in SortedCompliantSpaces) writeln(Space) end. dimension_non_compliance_check => findall(Space, (entity(Space), has_length(Space, Length), has_width(Space, Width), (less_than(Length, min_dimension_requirement); less_than(Width, min_dimension_requirement)))) = NonCompliantSpaces, sort(NonCompliantSpaces) = SortedNonCompliantSpaces, writeln("Habitable spaces not compliant with the building code requirement: "), foreach(Space in SortedNonCompliantSpaces) writeln(Space) end.</pre>

Table 9
Distribution of the number of further prompts needed in crash report translation.

Number of further prompts	0	1	2	3
Count	14	3	2	1

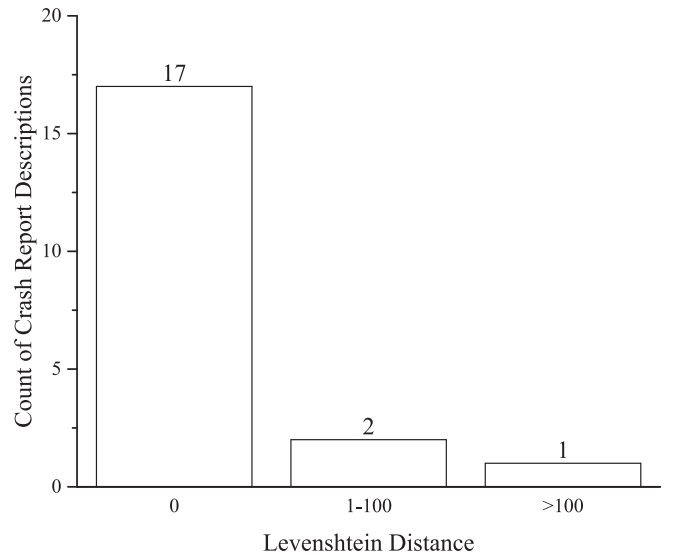


Fig. 21. Distribution of Levenshtein Distance in crash report translation experiments.

different granularity levels in the crash report translation experiments. Notably, after the initial two trials, the translator consistently delivered codes that accurately mirrored the crash report content and were free of

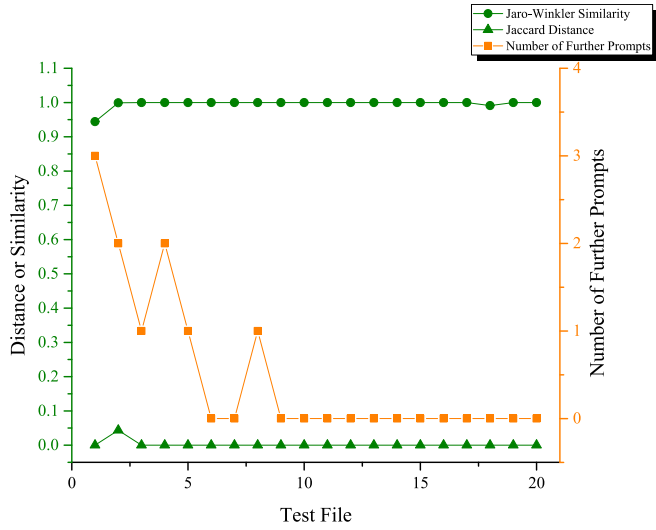


Fig. 22. Jaro Winkler Similarity, Jaccard Distance, and number of further prompts in crash report translation experiments.

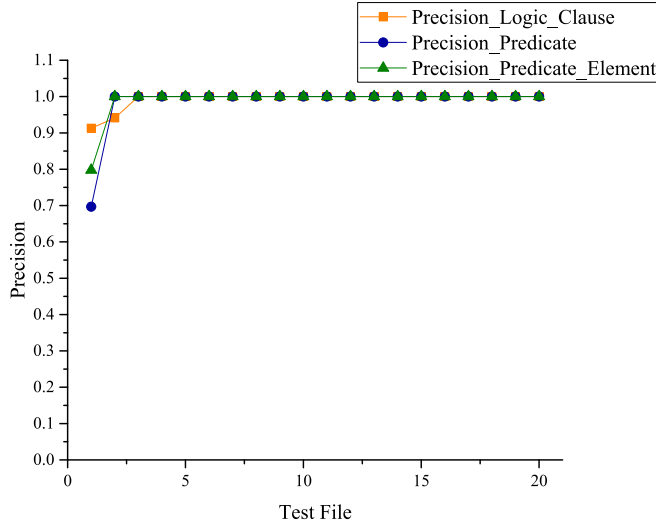


Fig. 23. Precision of crash report translation experiments.

grammatical errors, achieving optimal performance. An intriguing observation from the 18th experiment is its precision and recall scores of 1.00, accompanied by a Jaro-Winkler Similarity of 0.991. This discrepancy arises due to the necessity for reordering a few logic facts within the generated code, as Picat requires predicates sharing names to be defined consecutively for successful execution. The sensitivity of the Jaro-Winkler Similarity to string order explains why its value did not reach unity. Table 10 summarizes the average precision, recall, and F1 score for all cases, aligning with the Jaccard Distance and Jaro-Winkler Similarity to underscore the translator's good performance, thereby affirming the method's versatility.

The code generation times for the last ten experiments were documented, with an average time of 60.8 s per case. This marks a significant efficiency improvement over both manual and rule-based methods from previous research [47], reducing the time from 321.2 s per rule with the manual approach and 84.2 s per rule with the rule-based approach to 60.8 s per rule (where the code corresponds to each regulatory rule comprises multiple logic facts and two logic rules). This represents an 81.1 % and 27.8 % reduction in time, respectively, as shown in Fig. 25.

Table 10 presents a summary of the model's performance on the crash report translation task, highlighting near-perfect similarity and

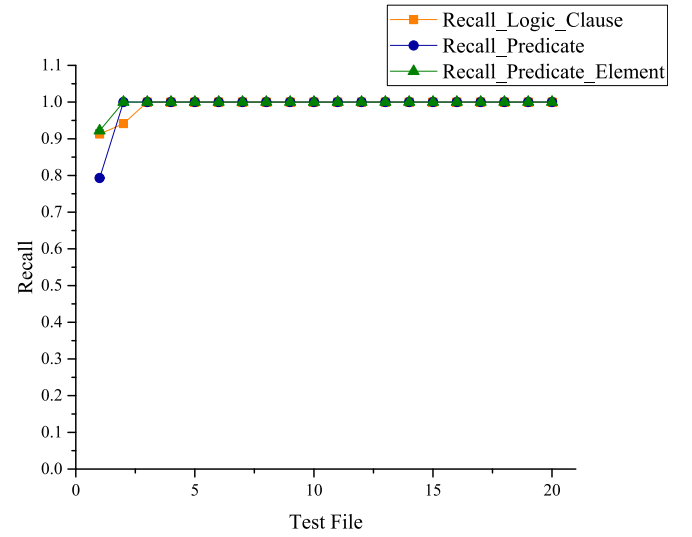


Fig. 24. Recall of crash report translation experiments.

Table 10

Evaluation metrics for crash report translation experiments.

Metrics	Mean	Standard deviation	Confidence intervals (95 %)
Jaro-Winkler similarity	0.9967	0.0125	[0.9909, 1.0000]
Jaccard distance	0.0022	0.0097	[0.0000, 0.0067]
Precision (logic clause)	0.9927	0.0229	[0.9820, 1.0000]
Recall (logic clause)	0.9927	0.0229	[0.9820, 1.0000]
Precision (predicate)	0.9848	0.0678	[0.9531, 1.0000]
Recall (predicate)	0.9897	0.0463	[0.9680, 1.0000]
Precision (predicate element)	0.9899	0.0452	[0.9687, 1.0000]
Recall (predicate element)	0.9961	0.0174	[0.9879, 1.0000]

minimal dissimilarity, along with high precision and recall across all evaluation levels. The Jaro-Winkler Similarity metric boasts a mean score of 0.9967 and a very low standard deviation of 0.0125. The 95 % confidence interval of [0.9909, 1.0000] underscores the consistently high similarity between the translated texts and the gold standard across the dataset. Complementing this, the Jaccard Distance metric reveals a

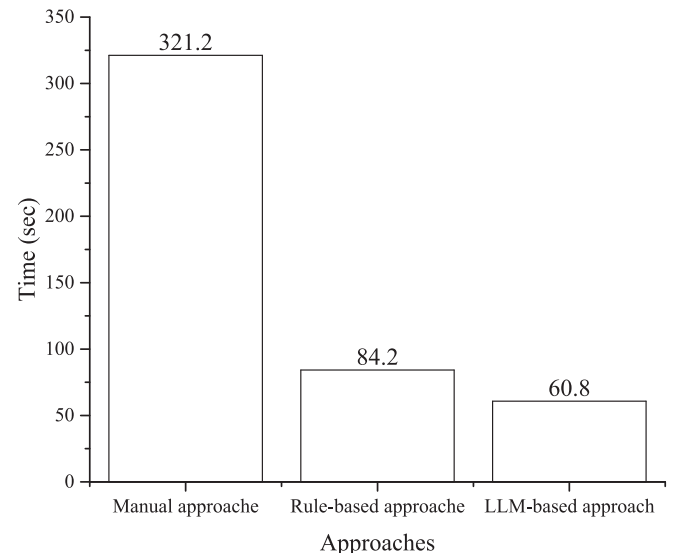


Fig. 25. Time comparison between different approaches.

mean score of 0.0022 with a standard deviation of 0.0097, pointing to minimal dissimilarity. This is further corroborated by the 95 % confidence interval [0.0000, 0.0067], which highlights the model's capability to preserve the semantic content of the original texts with very little variation. In terms of precision and recall, the model exhibits outstanding performance. For logic clauses, the mean precision, recall, and F1 score are all 0.9927, reflecting the model's proficiency in accurately and comprehensively capturing logic components. At the predicate level, the model continues to perform strongly, with mean precision at 0.9848, recall at 0.9897, and F1 score at 0.9872, indicating high accuracy in identifying and translating predicates. The highest scores are seen at the predicate element level, where the model achieves a mean precision of 0.9899, recall of 0.9961, and an F1 score of 0.9930. This showcases the model's exceptional accuracy and consistency in handling detailed elements. These high precision and recall scores, coupled with low standard deviations and narrow confidence intervals, underscore the model's robustness and reliability. The findings indicate that the model not only captures relevant information accurately but does so with minimal variability, making it highly effective for translating crash reports with high fidelity and consistency.

5.3. Comparison of the building code translation and the crash report translation

Since the same evaluation metrics were used in both the building code translation task and the crash report translation task, the translator's performance across these two experiments was compared using *t*-tests and Cohen's *d* metric [87]. The *t*-test was employed to compare the means of the performance metrics between the two tasks, helping to determine whether the differences in the model's performance are statistically significant. To ensure the appropriate version of the *t*-test was used, Levene's test was conducted beforehand to assess the equality of variances of each metric between the two tasks. The results of Levene's test determined whether the *t*-test should assume equal or unequal variances. For both the *t*-test and Levene's test, a low *p*-value (typically <0.05) indicates a significant difference, whereas a high *p*-value suggests no significant difference. Additionally, Cohen's *d* was used to measure the effect sizes of each metric between the two tasks, indicating the magnitude of the difference [87]. A higher absolute value of Cohen's *d* indicates a larger effect size, with thresholds generally defined as 0.2 (small), 0.5 (medium), and 0.8 (large) [87].

Table 11 provides a detailed comparison of the model's performance in the two experiments. The results show significant differences in Levene's test for Jaro-Winkler similarity and the logic clause level recall, indicating the need of using unequal variance *t*-test. The results from this unequal variance *t*-test then showed significant differences in means between the two tasks, as reflected in the large effect sizes of Cohen's *d* for Jaro-Winkler similarity and the logic clause level recall. The crash report translation results outperformed the building code translation results, showing variations in the model performance in different types

Table 11
Comparison of statistical significance between two experiments.

Metrics	<i>P</i> value of Levene's test	<i>P</i> value of <i>t</i> -test	Cohen's <i>d</i> (effect sizes)
Jaro-Winkler similarity	9.5045E-10	1.8146E-07	-1.0436
Jaccard distance	0.0876	0.0876	0.4586
Precision (logic clause)	0.0870	0.0870	-0.4595
Recall (logic clause)	0.0336	0.0028	-0.5739
Precision (predicate)	0.4285	0.4285	-0.2108
Recall (predicate)	0.1373	0.1373	-0.3979
Precision (predicate element)	0.4517	0.4517	-0.2003
Recall (predicate element)	0.0739	0.0739	-0.4803

of tasks.

For other metrics, such as Jaccard distance and precision and recall at the predicate and predicate element levels, the *t*-test results did not show significant differences in means, although some medium effect sizes of Cohen's *d* suggest that there might be some differences but not at the significant level.

It is worth noting that consistency was found between Levene's and *t*-tests results, with the *p*-values of these two tests staying the same or very close for most metrics (except for the recall at the logic clause level, as shown in Table 11). This could be due to the following reasons: In some cases, the data may be highly homogeneous within each group, leading to similar variances and means. For example, for Recall (predicate element), a large number of values in both groups are "1", causing less variation and more similar distributions between these two groups.

Overall, the analysis suggests that the model's prior engagement with building code translation and subsequent refinement operations positively impacted its performance on crash report translation, especially in terms of similarity and recall for logic clauses. The translator's enhanced performance in crash report translations, relative to building code translations, can be attributed to two main factors. Firstly, the narrative nature of crash reports presents a simpler and more comprehensible format than the complex jargon of building codes, facilitating the translator's comprehension and quantitative data extraction. Secondly, the prior engagement with building code translations has acclimatized the translator to Picat's syntax and task objectives, paving the way for improved performance in subsequent tasks.

5.4. Zero-shot scenario

Leveraging its sophisticated natural language processing capabilities, the GPT model has demonstrated its proficiency in translating complex programming directives into functional codes. It excels across a broad spectrum of coding tasks, including debugging and optimizing pre-existing code, as well as generating new algorithms [48–50]. Given the GPT model's prowess in code generation, the inquiry arises whether it could be applied directly to building code translation tasks in the absence of training data. To explore this, a zero-shot learning scenario was devised, where only background information of Picat and a specific task description were supplied to the model. For comparative purposes, the building code requirement listed in Table 5 was translated under this scenario using an identical prompt. The output generated by the GPT model, devoid of example rules (i.e., training data), is presented in Table 12. The model formulated a fact regarding the maximum permissible sprinkler count in a corridor, alongside a rule for assessing corridor designs against building code standards. Nonetheless, the model's output was marred by several deficiencies: 1) an inability to conduct numeric comparisons incorporating units or perform unit conversions, 2) incomplete coverage of the requirement's concepts and relationships, 3) a lack of logic rules for identifying noncompliance, 4) the absence of designed instances for evaluating the logic rule, and 5) grammatical inaccuracies, such as the misuse of ":-". Consequently, the translator, when deprived of training data, yields code that falls short in both functional and grammatical checks. This suggests that attaining functionality equivalent to manually crafted code would necessitate

Table 12
Generated code of the translator without training data.

Building code requirement	Generated code
"Where the design area of the sprinkler system consists of a corridor protected by one row of sprinklers, the maximum number of sprinklers required to be calculated is 13."	<i>max_sprinklers_for_corridor(13).</i>
[44]	<i>corridor_design_meets_code</i> (NumberOfSprinklers):- <i>max_sprinklers_for_corridor</i> (MaxSprinklers), NumberOfSprinklers=< MaxSprinklers.

multiple prompt iterations, thereby escalating manual intervention. This experiment proves that without specific training data, the GPT model encounters difficulties in translating domain-specific information, aligning with the conclusions of preceding studies [12,51]. This shows the value of the proposed framework and devised training data.

6. Contributions, ACC advancements, and practical benefits for the AEC industry

6.1. Contributions to the body of knowledge

The study in this paper contributes to the body of knowledge in three main aspects.

The primary contribution of this research is a novel prompt-based framework for building code information transformation. This approach enhances the precision and adaptability of information transformation while substantially reducing the requisite manual labor. The paper demonstrates the framework's effectiveness, versatility, and efficiency through rigorous evaluation. Unlike traditional rule-based methods, which demand extensive manual rule development, the proposed framework achieves similar levels of accuracy without such labor-intensive prerequisites. Furthermore, in comparison with machine learning-based methods, it requires minimal data annotation and relies on less training data, yet delivers superior accuracy. This effectively combines the high precision of rule-based approaches with the adaptability of machine learning approaches, presenting a more balanced and efficient solution for ACC.

The second major contribution is the validation of the developed framework's versatility, demonstrated through its application beyond building code information transformation and extended to the domain of crash report data translation. This broad applicability suggests its potential for aiding in the reconstruction of vehicle accidents and enhancing the safety assessment of autonomous vehicles. By successfully transforming crash report narratives, the study indicates the framework's capability to facilitate information transformation across various domains, highlighting its flexibility and scalability. This versatility is particularly important as it suggests that the framework can be adapted to other domain contexts, providing a universal tool for information transformation that could benefit multiple industries.

Last but not least, this research has produced a valuable dataset that codifies building regulations into a logic programming language, specifically addressing the quantitative requirements of Chapter 9 in the IBC 2015 [44]. This dataset not only enriches the existing pool of resources but also presents new opportunities for downstream tasks such as automated compliance checking and logic clause verification. The creation of this dataset marks a substantial advancement in the field, providing a foundation for further research and development in ACC. It opens up new possibilities for improving the accuracy and efficiency of compliance checking systems, thereby contributing to safer and more reliable AEC practices.

6.2. Contributions to the ACC literature

Building on these foundational contributions to the body of knowledge, the study also makes significant advancements in the field of automated code compliance checking (ACC).

First, the proposed framework significantly enhances the ACC process by automating the translation of building codes into machine-readable formats. This automation not only improves efficiency but also ensures that compliance checks are thorough and consistent, reducing the likelihood of human error. The framework's ability to accurately interpret complex building codes and transform them into executable rules ensures that all regulatory requirements are met precisely, which is critical for maintaining safety and quality standards in construction projects.

Second, the framework's demonstrated success in transforming crash

report data indicates its potential beyond just building codes. This broad applicability suggests that the framework could be a valuable tool for various regulatory compliance tasks across different industries. Its scalability allows it to handle diverse and evolving datasets, making it a robust solution for large-scale applications. This adaptability is particularly beneficial in the ACC literature, as it provides a flexible approach that can be tailored to diverse domain contexts and requirements.

Third, the development of a dataset that codifies building regulations into a logic programming language provides a valuable resource for future research and development in ACC. This dataset enables more precise and automated compliance verification, supporting the creation of more advanced compliance checking systems. In the ACC literature, this contribution highlights the importance of data-driven approaches in enhancing the accuracy and reliability of compliance checking, paving the way for more sophisticated and automated systems.

6.3. Practical benefits to the AEC industry

In real-world applications, the savings and efficiency gains from the proposed framework translate into numerous practical benefits for the AEC industry.

First, the proposed automated framework accelerates the compliance checking process. Traditional manual compliance checking is time-consuming; for example, the average time for building permit issuance is 79 days as of July 2024 [56]. The proposed method has clear advantages over manual interpretation of building regulations, with an average time of 60.8 s per rule compared to 321.2 s per rule. This leads to quicker project approvals and reduced delays. With this time efficiency, projects can progress without the bottlenecks associated with manual compliance checks, ensuring timely completion. Faster compliance checks allow for more accurate scheduling and resource allocation, improving overall project management.

Second, the framework offers substantial cost savings by reducing labor and training expenses. Existing compliance checking software, such as Solibri, does not allow users to create custom rules from scratch. These software solutions are therefore not always up-to-date with the latest local building codes and standards. The proposed framework minimizes the need for large teams of compliance officers or specialists. Additionally, the automated system reduces the need for continuous training, as updates to building codes can be integrated automatically. This translates to significant cost savings on both labor and training, making resource allocation more effective.

Third, the framework ensures a high level of accuracy in translating building codes, which directly impacts the quality of construction projects. Manual interpretation of building codes is prone to human errors, leading to non-compliance and costly rework. The framework ensures a high level of accuracy in translating building codes, minimizing human errors. This results in fewer instances of non-compliance and reduces the need for rework, saving both time and money. The consistent and precise application of building codes enhances the quality and safety of construction projects, leading to more reliable outcomes.

7. Conclusions

Automated code compliance checking is a critical task in the architecture, engineering, and construction domain, necessitating accurate representation of building codes in computable formats. These codes are inherently complex, subject to changes across different jurisdictions and over time. Existing methods for code transformation, despite their effectiveness, often require intensive manual input or a vast collection of annotated data, which may be impractical or cost prohibitive within the AEC sector.

A prompt-based framework was introduced in this paper, designed to automate the transformation of building code into a logic program, addressing the challenges of automated code compliance checking within the AEC industry. Specifically, this paper attempts to answer the

following three research questions:

1. How can the strengths of machine learning and rule-based approaches be effectively combined to improve building code representation for ACC?

To answer this question, the proposed framework integrates GPT models with prompt engineering to leverage the rule-based system's precision while benefiting from the flexibility of machine learning. The prompt-based approach allows the GPT model to understand and translate the complex and nuanced language of building codes into machine-readable formats. This synergy results in a system that can adapt to evolving building codes and diverse domain contexts, providing a robust and versatile solution for ACC.

2. How good are GPT models, when enhanced with prompt engineering, in translating complex building codes into machine-readable formats?

First, GPT models are trained on vast datasets, which makes them well-suited for interpreting the complex and nuanced language inherent in building codes. Second, the adaptability of GPT models allows them to be trained with additional data specific to regional and contextual nuances in building codes. This adaptability ensures they remain effective across diverse and evolving regulatory contexts. Third, the prompt-based approach enhances the model's ability to accurately and efficiently translate building codes into machine-readable formats with fewer manual inputs and less training data. This results in high precision and recall and demonstrates the framework's effectiveness in automated code transformation.

3. How much improvement in terms of accuracy and efficiency can be achieved compared to pure rule-based or machine learning-based methods?

The framework's performance was assessed through two distinct tasks: building code translation and crash report translation. In the building code translation task, the framework demonstrated high precision (97.37 %) and recall (95.88 %) at the logic clause level. This performance is superior to the existing rule-based and machine learning-based methods. Furthermore, unlike rule-based methods that require extensive manual rule development and machine learning-based methods that require extensive annotated data, the proposed framework performs well with a much smaller proportion (2 %) of training data. This reduces the time and cost associated with data preparation and model training, enhancing the framework's practicality for real-world applications. The subsequent application to crash report translation further underscored the framework's adaptability and scalability, showcasing improved performance and efficiency, with a 27.8 % reduction in code generation time compared to existing rule-based methods.

8. Generalizability, limitations and future work

The findings of this paper highlight significant advancements in the automation of ACC (i.e., automation in transformation of building codes into computable logic programs) through the integration of GPT models and prompt engineering. While the results are promising, several factors influence the generalizability of these findings. First, the framework has demonstrated adaptability and scalability by handling international building codes and car crash reports. However, building codes vary significantly across regions and countries. The framework's ability to accurately interpret and convert these diverse codes without extensive region-specific training remains to be tested. Second, the high precision and recall achieved in the building code translation task with only 2 % of training data suggest its high efficiency. However, the framework's performance in zero-shot scenarios, when no prior training data is provided, indicates that its effectiveness is closely tied to the availability of domain-specific training data. In regions where annotated data is extremely scarce or costly, this dependence could limit the framework's

generalizability. Third, the practical implementation of the framework in real-world scenarios depends on its integration with existing ACC systems. Compatibility with current software, ease of deployment, and the learning curve for users could impact its broader adoption and effectiveness. Fourth, although the framework reduces the need for extensive manual input, it still requires prompt engineering to adapt the GPT model to specific regulatory contexts. This necessity for expert involvement may still constrain the framework's generalizability, depending on the availability and expertise of individuals capable of performing these tasks. Lastly, while the framework has shown competitive performance compared to rule-based and machine learning-based methods, further comparative studies involving a broader range of the state-of-the-art ACC methods and different regulatory domain contexts are needed to further validate the results.

The following limitations are acknowledged in the current study. First, the focus has been primarily on translating quantitative information from building codes and crash reports, suggesting the need of future work for a broader examination of different information types, such as existential and qualitative requirements in building codes [52], for a more comprehensive understanding of the framework's capabilities. Second, while the framework has been tested on crash reports, specific challenges were encountered during the transformation process. These challenges include handling unstructured and variable data formats and ensuring the accuracy of extracted information. The framework's applicability and performance in diverse natural language processing tasks beyond building code transformation and crash report transformation remains to be fully explored in future research. Third, the reliance on a prompt-based methodology means that while the outputs generated are highly interpretable in relation to the specific prompts provided, the underlying mechanisms of the model's decision-making process remain opaque, which in the future may be enhanced with explainable AI techniques. Fourth, the use of large language models for the framework introduces limitations related to computational resources and scalability. LLMs' development requires substantial processing power, memory, and energy, which can translate to significant costs. Additionally, the complexity of network data handling and model maintenance further complicates large-scale deployments based on LLMs, and may cause data security and privacy concerns as well. Future research should explore more efficient algorithms and techniques to address such issues and improve scalability (e.g., through local implementation of small but effective LLMs), making the framework more accessible for widespread use in the AEC industry. Finally, the prompt-based methodology, while effective, introduces potential variability in responses from the language model. This variability can affect the consistency and reliability of the outcomes, especially when dealing with complex and nuanced building code requirements. Future improvements might involve the development of optimized prompt templates and advanced prompt engineering techniques (e.g., retrieval augmented generation) to further enhance consistency and reliability in the model's responses.

Declaration of use of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used OpenAI GPT4 to improve readability and language. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRediT authorship contribution statement

Fan Yang: Writing – original draft, Methodology. **Jiansong Zhang:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the research presented in this paper.

Data availability

All data that support the findings of this study are available upon reasonable request.

Acknowledgements

The authors would like to thank the National Science Foundation (NSF). This material is based on work supported by the NSF under Grant No. 1827733. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

References

- [1] F. Yang, J. Zhang, Y. Chen, L. Debs, A new schema of logic representation and reasoning for automated building code compliance checking, in: Proceedings of the GPEA Polytechnic Summit 2022: Session Papers, 2022. <https://par.nsf.gov/biblio/10419685-new-schema-logic-representation-reasoning-automated-building-code-compliance-checking> (accessed January 4, 2024).
- [2] C. Eastman, J. Lee, Y. Jeong, J. Lee, Automatic rule-based checking of building designs, *Autom. Constr.* 18 (2009) 1011–1033, <https://doi.org/10.1016/j.autcon.2009.07.002>.
- [3] J. Dimyadi, R. Amor, Automated Building Code Compliance Checking - Where is it at?, 2013, <https://doi.org/10.13140/2.1.4920.4161>.
- [4] R. Amor, J. Dimyadi, The promise of automated compliance checking, *Develop. Built Environ.* 5 (2021) 100039, <https://doi.org/10.1016/j.dibe.2020.100039>.
- [5] R. Zhang, N. El-Gohary, A deep neural network-based method for deep information extraction using transfer learning strategies to support automated compliance checking, *Autom. Constr.* 132 (2021) 103834, <https://doi.org/10.1016/j.autcon.2021.103834>.
- [6] B. Zhong, X. Xing, H. Luo, Q. Zhou, H. Li, T. Rose, W. Fang, Deep learning-based extraction of construction procedural constraints from construction regulations, *Adv. Eng. Inform.* 43 (2020) 101003, <https://doi.org/10.1016/j.aei.2019.101003>.
- [7] X. Wang, N. El-Gohary, Deep Learning-based named entity recognition and resolution of referential ambiguities for enhanced information extraction from construction safety regulations, *J. Comput. Civ. Eng.* 37 (2023) 04023023, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0001064](https://doi.org/10.1061/(ASCE)CP.1943-5487.0001064).
- [8] J. Zhang, N.M. El-Gohary, Semantic NLP-based information extraction from construction regulatory documents for automated compliance checking, *J. Comput. Civ. Eng.* 30 (2016) 04015014, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000346](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000346).
- [9] P. Zhou, N. El-Gohary, Ontology-based automated information extraction from building energy conservation codes, *Autom. Constr.* 74 (2017) 103–117, <https://doi.org/10.1016/j.autcon.2016.09.004>.
- [10] R. Ren, J. Zhang, Semantic rule-based construction procedural information extraction to guide jobsite sensing and monitoring, *J. Comput. Civ. Eng.* 35 (2021) 04021026, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000971](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000971).
- [11] J. Zhang, N.M. El-Gohary, Semantic-based logic representation and reasoning for automated regulatory compliance checking, *J. Comput. Civ. Eng.* 31 (2017) 04016037, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000583](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000583).
- [12] J. Zhang, How Can ChatGPT Help in Automated Building Code Compliance Checking?, in: ISARC. Proceedings of the International Symposium on Automation and Robotics in Construction IAARC Publications, Waterloo, Canada, 2023, pp. 63–70. <https://www.proquest.com/docview/2841146160/abstract/DCACFD1BA48643B9PQ/1> (accessed January 4, 2024).
- [13] X. Chen, J. Ye, C. Zu, N. Xu, R. Zheng, M. Peng, J. Zhou, T. Gui, Q. Zhang, X. Huang, How Robust is GPT-3.5 to Predecessors? A Comprehensive Study on Language Understanding Tasks, 2023, <https://doi.org/10.48550/arXiv.2303.00293>.
- [14] M.O. Topal, A. Bas, I. van Heerden, Exploring Transformers in Natural Language Generation: GPT, BERT, and XLNet, 2021, <https://doi.org/10.48550/arXiv.2102.08036>.
- [15] R. Sawai, I. Paik, A. Kuwana, Sentence augmentation for language translation using GPT-2, *Electronics* 10 (2021) 3082, <https://doi.org/10.3390/electronics10243082>.
- [16] B. Banar, S. Colton, A systematic evaluation of GPT-2-based music generation, in: T. Martins, N. Rodríguez-Fernández, S.M. Rebelo (Eds.), *Artificial Intelligence in Music, Sound, Art and Design*, Springer International Publishing, Cham, 2022, pp. 19–35, https://doi.org/10.1007/978-3-031-03789-4_2.
- [17] K. Crowston, E.E. Allen, R. Heckman, Using natural language processing technology for qualitative data analysis, *Int. J. Soc. Res. Methodol.* 15 (2012) 523–543, <https://doi.org/10.1080/13645579.2011.625764>.
- [18] D.D. Lewis, K.S. Jones, Natural language processing for information retrieval, *Commun. ACM* 39 (1996) 92–101, <https://doi.org/10.1145/234173.234210>.
- [19] D. Dessi, F. Osborne, D. Reforgiato Recupero, D. Buscaldi, E. Motta, SCICERO: a deep learning and NLP approach for generating scientific knowledge graphs in the computer science domain, *Knowl.-Based Syst.* 258 (2022) 109945, <https://doi.org/10.1016/j.knsys.2022.109945>.
- [20] H. Luo, P. Gong, A BIM-based code compliance checking process of deep foundation construction plans, *J. Intell. Robot. Syst.* 79 (2015) 549–576, <https://doi.org/10.1007/s10846-014-0120-z>.
- [21] X. Xu, H. Cai, Semantic approach to compliance checking of underground utilities, *Autom. Constr.* 109 (2020) 103006, <https://doi.org/10.1016/j.autcon.2019.103006>.
- [22] Z. Zhao, S. Song, B. Duah, J. Macbeth, S. Carter, M.P. Van, N.S. Bravo, M. Klenk, K. Sick, A.L.S. Filipowicz, More Human than Human: LLM-Generated Narratives Outperform Human-LLM Interleaved Narratives, in: Proceedings of the 15th Conference on Creativity and Cognition, Association for Computing Machinery, New York, NY, USA, 2023, pp. 368–370, <https://doi.org/10.1145/3591196.3596612>.
- [23] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, B. Myers, Using an LLM to Help with Code Understanding, *IEEE Computer Society*, 2024, p. 881, in: <https://www.computer.org/csdl/proceedings-article/icse/2024/021700a881/1V5BksOlzJm> (accessed March 17, 2024).
- [24] M. Kazemitabar, X. Hou, A. Henley, B.J. Ericson, D. Weintrop, T. Grossman, How Novices Use LLM-Based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment, in: Proceedings of the 23rd Koli Calling International Conference on Computing Education Research, Association for Computing Machinery, New York, NY, USA, 2024, pp. 1–12, <https://doi.org/10.1145/3631802.3631806>.
- [25] S. Ding, Z. Liu, X. Dong, P. Zhang, R. Qian, C. He, D. Lin, J. Wang, SongComposer: A Large Language Model for Lyric and Melody Composition in Song Generation, 2024, <https://doi.org/10.48550/arXiv.2402.17645>.
- [26] L. Qu, S. Wu, H. Fei, L. Nie, T.-S. Chua, LayoutLLM-T2I: Eliciting Layout Guidance from LLM for Text-to-Image Generation, in: Proceedings of the 31st ACM International Conference on Multimedia, Association for Computing Machinery, New York, NY, USA, 2023, pp. 643–654, <https://doi.org/10.1145/3581783.3612012>.
- [27] R. Kowalski, Logic Programming, in: J.H. Siekmann (Ed.), *Handbook of the History of Logic*, North-Holland, 2014, pp. 523–569, <https://doi.org/10.1016/B978-0-444-51624-4.50012-5>.
- [28] F. Nogatz, D. Seipel, Implementing GraphQL as a query language for deductive databases in SWI-prolog using DCGs, quasi quotations, and dicts, *Electron. Proc. Theor. Comput. Sci.* 234 (2017) 42–56, <https://doi.org/10.4204/EPTCS.234.4>.
- [29] S. Ceri, G. Gottlob, L. Tanca, Logic programming and databases: An overview, in: S. Ceri, G. Gottlob, L. Tanca (Eds.), *Logic Programming and Databases*, Springer, Berlin, Heidelberg, 1990, pp. 1–15, https://doi.org/10.1007/978-3-642-83952-8_1.
- [30] F. Rossi, Constraint satisfaction problems in logic programming, *SIGART Bull.* (1988) 24–28, <https://doi.org/10.1145/54350.54352>.
- [31] N.-F. Zhou, Programming finite-domain constraint propagators in action rules, *Theory Pract. Logic Progr.* 6 (2006) 483–507, <https://doi.org/10.1017/S1471068405002590>.
- [32] N.-F. Zhou, H. Kjellerstrand, J. Fruhman, *Constraint Solving and Planning with Picat*, Springer International Publishing, Cham, 2015, <https://doi.org/10.1007/978-3-319-25883-6>.
- [33] A. Hliaoutakis, G. Varelakis, E. Voutsakis, E.G.M. Petrakis, E. Milios, Information retrieval by semantic similarity, *IJISWIS* 2 (2006) 55–73, <https://doi.org/10.4018/jswis.2006070104>.
- [34] M. Eminagaoglu, A new similarity measure for vector space models in text classification and information retrieval, *J. Inf. Sci.* 48 (2022) 463–476, <https://doi.org/10.1177/0165551520968055>.
- [35] H. Arabi, M. Akbari, Improving plagiarism detection in text document using hybrid weighted similarity, *Expert Syst. Appl.* 207 (2022) 118034, <https://doi.org/10.1016/j.eswa.2022.118034>.
- [36] A. Das, D. Saha, Deep learning based Bengali question answering system using semantic textual similarity, *Multimed. Tools Appl.* 81 (2022) 589–613, <https://doi.org/10.1007/s11042-021-11228-w>.
- [37] J. Žizka, F. Dařena, Automatic sentiment analysis using the textual pattern content similarity in natural language, in: P. Sojka, A. Horák, I. Kopeček, K. Pala (Eds.), *Text, Speech and Dialogue*, Springer, Berlin, Heidelberg, 2010, pp. 224–231, https://doi.org/10.1007/978-3-642-15760-8_29.
- [38] A. Svyatkovskiy, S.K. Deng, S. Fu, N. Sundaresan, IntelliCode Compose: Code Generation Using Transformer, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1433–1443, <https://doi.org/10.1145/3368089.3417058>.
- [39] G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, S. Gulwani, SynchroMesh: Reliable Code Generation from Pre-trained Language Models, 2022, <https://doi.org/10.48550/arXiv.2201.11227>.
- [40] J. Liu, C.S. Xia, Y. Wang, L. Zhang, Is your code generated by ChatGPT really correct? Rigorous Evaluation of large language models for code generation, *Adv. Neural Inform. Proc. Syst.* 36 (2023) 21558–21572, <https://doi.org/10.48550/arXiv.2305.01210>.
- [41] Efficient Implementation of the Levenshtein-Algorithm, Fault-tolerant Search Technology, Error-tolerant Search Technologies. <http://www.levenshtein.net/> (accessed March 17, 2024).

- [42] S. Kosub, A note on the triangle inequality for the Jaccard distance, *Pattern Recogn. Lett.* 120 (2019) 36–38, <https://doi.org/10.1016/j.patrec.2018.12.007>.
- [43] Y. Wang, J. Qin, W. Wang, Efficient approximate entity matching using Jaro-Winkler distance, in: A. Bouguettaya, Y. Gao, A. Klimenko, L. Chen, X. Zhang, F. Dzerzhinskiy, W. Jia, S.V. Klimenko, Q. Li (Eds.), *Web Information Systems Engineering – WISE 2017*, Springer International Publishing, Cham, 2017, pp. 231–239, https://doi.org/10.1007/978-3-319-68783-4_16.
- [44] International Code Council (Ed.), *International Building Code 2015 IBC*, International Code Council, Country Club Hills, Ill, 2014 (ISBN: 978-1609834685).
- [45] Michigan Traffic Crash Facts. <https://www.michigantrafficcrashfacts.org/> (accessed March 17, 2024).
- [46] Y. Li, J. Shi, Z. Zhang, A Novel Approach for Rapid Development Based on ChatGPT and Prompt Engineering, 2023, <https://doi.org/10.48550/arXiv.2312.13115>.
- [47] J. Wu, J. Zhang, Model validation using invariant signatures and logic-based inference for automated building code compliance checking, *J. Comput. Civ. Eng.* 36 (2022) 04022002, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0001002](https://doi.org/10.1061/(ASCE)CP.1943-5487.0001002).
- [48] M.A. Haque, S. Li, The potential use of ChatGPT for Debugging and Bug Fixing, *Jultika.Oulu.Fi*, 2023. <https://oulurepo.oulu.fi/handle/10024/44572> (accessed March 17, 2024).
- [49] A. Narasimhan, K.P.A.V. Rao, V.M. B, CGEMS: A Metric Model for Automatic Code Generation using GPT-3, 2021, <https://doi.org/10.48550/arXiv.2108.10168>.
- [50] J.Y. Khan, G. Uddin, Automatic Code Documentation Generation Using GPT-3, in: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, Association for Computing Machinery, New York, NY, USA, 2023, pp. 1–6, <https://doi.org/10.1145/3551349.3559548>.
- [51] C. Li, X. Su, H. Han, C. Xue, C. Zheng, C. Fan, Quantifying the impact of large language models on collective opinion, *Dynamics* (2023), <https://doi.org/10.48550/arXiv.2308.03313>.
- [52] J. Zhang, N.M. El-Gohary, Automated information transformation for automated regulatory compliance checking in construction, *J. Comput. Civ. Eng.* 29 (2015) B4015001, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000427](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000427).
- [53] T.H. Beach, J.-L. Hippolyte, Y. Rezgui, Towards the adoption of automated regulatory compliance checking in the built environment, *Autom. Constr.* 118 (2020) 103285, <https://doi.org/10.1016/j.autcon.2020.103285>.
- [54] X. Xue, J. Zhang, Regulatory information transformation ruleset expansion to support automated building code compliance checking, *Autom. Constr.* 138 (2022) 104230, <https://doi.org/10.1016/j.autcon.2022.104230>.
- [55] T.H. Beach, Y. Rezgui, H. Li, T. Kasim, A rule-based semantic approach for automated regulatory compliance in the construction sector, *Expert Syst. Appl.* 42 (2015) 5219–5231, <https://doi.org/10.1016/j.eswa.2015.02.029>.
- [56] Average time for permit issuance. City of Chicago, in: *Secondary City of Chicago* (Ed.), *Secondary Average Time for Permit Issuance*, City of Chicago, 2020. <https://www.chicago.gov/city/en/depts/bldgs.html> (Last Access: July, 23rd, 2024).
- [57] L. Giray, Prompt engineering with ChatGPT: a guide for academic writers, *Ann. Biomed. Eng.* 51 (2023) 2629–2633, <https://doi.org/10.1007/s10439-023-03272-4>.
- [58] J.H. Garrett, M.E. Palmer, S. Demir, Delivering the Infrastructure for Digital Building Regulations, *J. Comp. Civil Eng.* 28 (2) (2014), [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000369](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000369).
- [59] X. Xue, J. Zhang, Y. Chen, Question-answering Framework for Building Codes Using Fine-tuned and Distilled Pre-trained Transformer Models, *Automation in Construction* 168 (Part A) (2024) 105730, <https://doi.org/10.1016/j.autcon.2024.105730>.
- [60] J. Zhang, F. Yang. Building a Bridge between Building Information Modeling and Digital Twins: Introducing Invariant Signatures of Architecture, Engineering, and Construction Objects. *Chapter 2, Digital Twins in Construction and the Built Environment*, ASCE, Reston, VA, 2024, <https://doi.org/10.1061/97807848485606.ch2>.
- [61] F. Yang, J. Zhang. Application of Graph Convolutional Networks to Classification of Building Code Requirements. *Construction Research Congress (CRC)*, ASCE, Reston, VA, 2024, pp. 836–845, <https://doi.org/10.1061/97807848485262.085>.
- [62] J. Wu, J. Zhang, New automated BIM object classification method to support BIM interoperability, *J. Comput. Civ. Eng.* 33 (5) (2019) 04019033, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000858](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000858).
- [63] J. Zhang, N. El-Gohary, Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking, *Automation in Construction* 73 (2017) 45–57, <https://doi.org/10.1016/j.autcon.2016.08.027>.
- [64] J. Wu, X. Xue, and J. Zhang. Invariant Signature, Logic Reasoning, and Semantic Natural Language Processing (NLP)-Based Automated Building Code Compliance Checking (I-SNACC) Framework. *Journal of Information Technology in Construction*, 28, Special Issue of The Eastman Symposium, 2023, 1-18. doi: 10.36680/j.itcon.2023.001.
- [65] X. Xue, J. Wu, Semi-Automated Generation of Logic Rules for Tabular Information in Building Codes to Support Automated Code Compliance Checking, *J. Comput. Civil Eng.* 36 (1) (2022) 04021033, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0001000](https://doi.org/10.1061/(ASCE)CP.1943-5487.0001000).
- [66] J. Wu, J. Zhang, Model Validation Using Invariant Signatures and Logic-Based Inference for Automated Building Code Compliance Checking, *J. Comput. Civil Eng.* 36 (3) (2022) 04022002, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0001002](https://doi.org/10.1061/(ASCE)CP.1943-5487.0001002).
- [67] X. Xue, Part-of-speech tagging of building codes empowered by deep learning and transformational rules, *Adv. Eng. Inform.* 47 (2021) 101235, <https://doi.org/10.1016/j.aei.2020.101235>.
- [68] J. Zhang, N. El-Gohary. A prototype system for fully automated code checking, *Osaka, Japan*, 2016, pp. 535–542.
- [71] W.W. Cohen, P. Ravikumar, S.E. Fienberg, A Comparison of String Distance Metrics for Name-Matching Tasks (2003).
- [72] OpenAI. GPT-4 Technical Report, arXiv.Org. <https://arxiv.org/abs/2303.08774v6>, 2023 (accessed October 7, 2024).
- [73] Python Release Python 3.9.10, Python.Org. <https://www.python.org/downloads/r/lease/python-3910/>, 2022 (accessed October 7, 2024).
- [74] An Exploration into Overlooked Areas of Forensics Data Analysis, Case Management and Metadata Using Automation and Natural Language Processing - ProQuest. <https://www.proquest.com/docview/3073244608>, 2023 (accessed October 7, 2024).
- [75] ai-admin, Discover the Cutting-Edge Breakthroughs and Advancements in Artificial Intelligence for 2022, AI for Social Good. <https://aiforsocialgood.ca/blog/discover-the-cutting-edge-breakthroughs-and-advancements-in-artificial-intelligence-for-2022>, 2024 (accessed October 7, 2024).
- [76] System Programming Lab Lecture 3 Eng Alia Hamwi. <https://slidetodoc.com/system-programming-lab-lecture-3-eng-alia-hamwi/> (accessed October 7, 2024).
- [77] M. Crispino, F.P. Conteduca. It's a match! Linking foreign counterparts in Italian customs data to their balance sheets, *Questioni di Economia e Finanza (Economic and Financial Issues)* 823, 2023. ISSN 1972-6643 (online).
- [78] E. Venner, D. Muzny, J.D. Smith, K. Walker, C.L. Neben, C.M. Lockwood, P. Empey, G.A. Metcalf, C. Kachulis, S. Mian, A. Musick, H.L. Rehm, S. Harrison, S. Gabriel, R.A. Gibbs, D. Nickerson, A.Y. Zhou, K. Doheny, B. Ozenberger, S. E. Topper, N.J. Lennon, The All of Us Research Program Regulatory Working Group, Whole-genome sequencing as an investigational device for return of hereditary disease risk and pharmacogenomic results as part of the All of Us Research Program, *Genome Medicine* 14 (2022) 34, <https://doi.org/10.1186/s13073-022-01031-z>.
- [79] F. Pouget, Distributed System of HoneyPot Sensors: Discrimination and Correlative Analysis of Attack Processes | Policy Commons. <https://policycommons.net/artifacts/15458188/distributed-system-of-honeypot-sensors/16351558/>, 2006 (accessed October 8, 2024).
- [80] P. Stefaniak, M. Stachowiak, W. Koperska, A. Skoczylas, P. Śliwiński, Application of Wearable Computer and ASR Technology in an Underground Mine to Support Mine Supervision of the Heavy Machinery Chamber, *Sensors* 22 (2022) 7628, <https://doi.org/10.3390/s22197628>.
- [81] O. Rozinek, J. Mareš, Fast and Precise Convolutional Jaro and Jaro-Winkler Similarity. <https://doi.org/10.23919/FRUCT61870.2024.10516360>.
- [82] R. Mohamed. Personal Names Records Detection And Linkage In Unstructured Dutch Text, *For Anonymisation*, 2021.
- [84] B. Zhong, X. Pan, P.E.D. Lov, J. Sun, Hazard analysis: A deep learning and text mining framework for accident prevention, *Adv. Eng. Inform.* 46 (2020) 101152, <https://doi.org/10.1016/j.aei.2020.101152>.
- [85] Binary codes capable of correcting deletions, insertions, and reversals, *Sov. Phys. Doklady* 10 (1966) 707–710. <https://ui.adsabs.harvard.edu/abs/1966SPhD...10..707L/abstract>.
- [86] A.C. Phan, Similarity algorithms for fuzzy joint computation in big data processing environment, *J. Comput. Sci. Cybernetics* 39 (2) (2023) 101–124, <https://doi.org/10.15625/1813-9663/17589>.
- [87] *Statistical Power Analysis for the Behavioural Sciences*, 2nd Edition, Academic Press Inc, New York.